VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Multidisciplinary Project

H.E.R.A

(Home Environment & Response Assistant)

Instructor: Dr. Võ Thị Ngọc Châu

Student's Name	Student ID
Nguyễn Tiến Hưng	2352441
Hồ Lâm Khánh Vy	2353353
Trần Anh Đức	2352271
Nguyễn Khánh Hưng	2352435

Ho Chi Minh City, January 2026.

Member list & Workload

Group Leader: Nguyễn Tiến Hưng - hung.nguyenk23cse@hcmut.edu.vn

No.	Full name	Contribution	Percentage
1.	Nguyễn Tiến Hưng		100%
2.	Hồ Lâm Khánh Vy		100%
3.	Trần Anh Đức		100%
4.	Nguyễn Khánh Hưng		100%

Lecturer's Assessment

No.	Full name	Evaluation	Notes
1.	Nguyễn Tiến Hưng		
2.	Hồ Lâm Khánh Vy		
3.	Trần Anh Đức		
4.	Nguyễn Khánh Hưng		

Contents

List of Figures

1 Introduction

1.1 Problem Statement

Context and Target Environment

The evolution of the "Smart Home" has transitioned from simple connectivity to the demand for intelligent automation. While the general market is saturated with fragmented IoT solutions, this project specifically targets modern urban households—such as mid-sized apartments or townhouses—occupied by busy families, elderly members, or pets. In these environments, standard smart platforms function merely as centralized remote controls. They enable manual actuation via smartphone applications but fail to deliver a cohesive, autonomous living experience tailored to the specific safety, convenience, and energy needs of an active urban family.

Key Challenges

Despite the ubiquity of connected hardware, several critical architectural and functional deficiencies persist in residential deployments:

- **Data Silos & Fragmentation:** Devices from different manufacturers often operate on proprietary protocols. This fragmentation prevents the aggregation of data into a single Unified Source of Truth, making holistic analytics and cross-device correlation impossible. [1]
- **Deterministic vs. Probabilistic Limitations:** Current systems rely on rigid, rule-based automation (e.g., "turn on lights at 6 PM"). They lack the probabilistic reasoning needed to adapt to dynamic human routines or distinguish between a human and a pet triggering a motion sensor.
- **Data Rich, Information Poor (DRIP):** IoT devices generate high-velocity telemetry daily. Without robust Data Engineering pipelines (ETL processes) to filter noise, users are overwhelmed with raw notifications rather than actionable insights regarding energy anomalies.
- **Reactive Operation & Energy Inefficiency:** Systems operate reactively (e.g., cooling a room only after it gets hot). This lag degrades user comfort and leads to significant energy waste, as the system cannot pre-emptively optimize resources.

Operational Assumptions & Constraints

To effectively address these challenges, the H.E.R.A system is designed with the following bounded parameters:

- **Physical Scope:** The system is optimized for a medium-sized residential house with an estimated area of approximately 100–150 square meters. Instead of focusing on a single enclosed room, H.E.R.A is designed to support multiple household zones such as the living room, bedrooms, kitchen, and common areas. The spatial model therefore focuses on house-level environmental monitoring, zone-based device control, and resident interaction across the home while keeping the deployment scope realistic for a residential prototype.
- **Infrastructure Compatibility:** The architecture assumes the presence of a stable Wi-Fi network for cloud-based AI processing (such as NLP and heavy predictive modeling). However, it requires local Edge processing capabilities (e.g., via ESP32) to maintain critical deterministic operations and sensor readings if internet connectivity drops.
- **Privacy & Security:** Given the continuous ingestion of environmental telemetry and voice commands, the system is constrained by strict data privacy requirements, ensuring that behavioral data and audio inputs are securely processed and shielded from unauthorized access.

The Need for a Solution and Scope of Intelligence

There is an urgent need for a comprehensive AIoT (AI + IoT) platform that bridges the gap between physical control and cognitive processing for urban homes. A truly intelligent system must transcend basic command execution to act as a proactive agent. H.E.R.A addresses this by narrowing its "smart" focus onto three prioritized pillars:

1. **AI-Enhanced Security:** Utilizing machine learning to differentiate between genuine human presence and pets, thereby eliminating false alarms and providing reliable monitoring.
2. **Proactive Energy Management:** Deploying predictive analytics to optimize power consumption based on historical usage patterns and environmental forecasts.
3. **Frictionless Interaction:** Leveraging Natural Language Processing (NLP) to allow residents to control their environment intuitively, effectively shifting the paradigm from manual "Smart Control" to autonomous "Smart Living."

1.2 Stakeholders and Key Users

To keep the project scope focused and consistent with the intended residential use case, the H.E.R.A system identifies only one core user group:

- **Residents:** Residents are the primary and only target users of the system. They live in and interact with the smart home environment on a daily basis. Their main concerns include comfort, convenience, health protection, energy awareness, and household safety. They use H.E.R.A to monitor environmental conditions, issue voice or text commands, receive alerts, and control connected devices across the house.

1.3 Solution

To fulfill the resident-centered business requirements, the H.E.R.A (Home Environment & Response Assistant) is a comprehensive AI-powered virtual assistant system designed to improve daily residential living. Unlike traditional passive smart homes, H.E.R.A integrates a robust IoT infrastructure with advanced Natural Language Processing (NLP) and Data Analytics. This fusion allows the system to continuously monitor environmental conditions across a medium-sized house while also understanding and executing resident commands via voice or text. By bridging the gap between raw telemetry and human intent, H.E.R.A delivers a living environment that is proactively energy-efficient, secure, and deeply personalized for residents.

System Workflow

The system operates on a dual-stream data pipeline, integrating both automated environmental monitoring and user-centric interaction into a unified architecture:

1. **Multi-modal Data Acquisition:** The system continuously ingests data from two primary sources:
   - Telemetry: Sensors monitor physical parameters (temperature, humidity, motion) and device states, transmitting data via MQTT.
   - User Interaction: A dedicated interface captures voice commands or text inputs from the user, serving as the direct communication bridge.
2. **Processing & Verification:** A centralized backend processes these incoming streams. For telemetry, it filters noise and verifies if data exceeds safety thresholds. Simultaneously, for user inputs, it initiates a Speech-to-Text (STT) process (if applicable) to convert audio signals into processable query strings.
3. **Intelligence Layer (NLP & Analytics):** This is the decision-making core:
   - Natural Language Understanding (NLU): AI models analyze user queries to extract intents (e.g., "turn on") and entities (e.g., "living room light").
   - Predictive Analytics: Machine learning algorithms analyze historical logs to forecast trends or detect anomalies, enriching the decision context.
4. **Action, Control & Feedback:** Based on either AI interpretation or threshold triggers, the system executes commands:
   - Device Control: Sending signals to microcontrollers (ESP32/YoLo) to adjust physical devices.
   - User Response: Generating a natural language response (Text-to-Speech) to confirm actions or report status back to the user.

5. **Activity Logging:** Every interaction—whether a sensor reading, a voice command, or a system automated response—is recorded in an activity log. This maintains a single source of truth for debugging, dashboard visualization, and future model training.

By combining reactive automation with interactive AI capabilities, this workflow fulfills the dual purpose of a robust monitoring system and an intelligent virtual assistant.

Architectural Hierarchy

To ensure scalability, maintainability, and a clear separation of concerns, the H.E.R.A system architecture is bifurcated into two distinct layers. This two-tier design decouples the physical hardware constraints from the high-level computational logic:

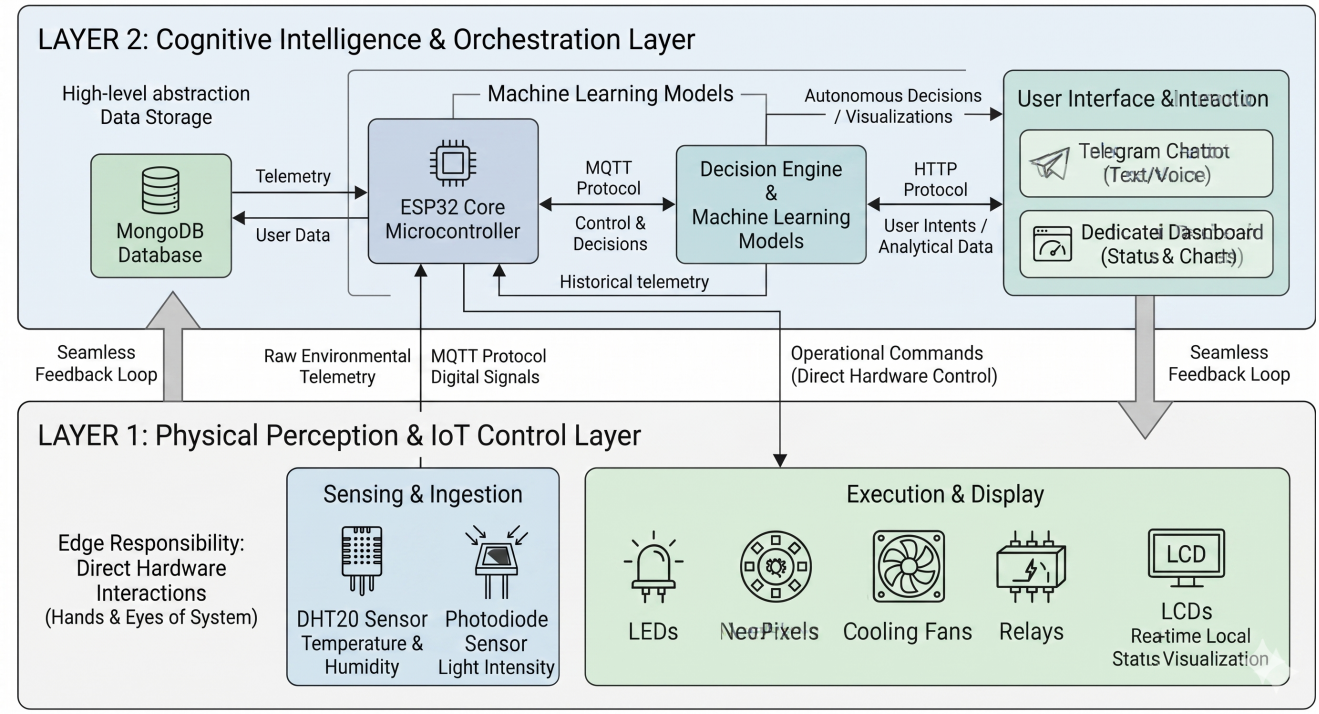

Figure 1.1: System Architecture of H.E.R.A

- **Layer 1: The Physical Perception & IoT Control Layer**
  This foundational layer serves as the direct interface with the physical environment. It is responsible for Data Acquisition and Device Actuation.
  - **Sensing & Ingestion:** Utilizing the DHT20 sensor, this layer captures raw environmental telemetry (specifically temperature and humidity) and converts physical phenomena into digital signals. These signals are transmitted to the core processing unit via the lightweight MQTT protocol to ensure stable, low-latency communication.
  - **Execution & Display:** This component receives operational commands from the upper layer and executes them through various actuators—including LEDs, NeoPixels, cooling fans, and relays—to physically alter the home environment. Additionally, it utilizes LCDs for real-time local status visualization.
  - **Edge Responsibility:** It handles immediate hardware interactions at the sensor/actuator level, acting as the reliable "hands and eyes" of the smart home system.
- **Layer 2: The Cognitive Intelligence & Orchestration Layer**
  Powered by an ESP32 core microcontroller acting as the main processing bridge and backed by a MongoDB database, this high-level layer abstracts the complexity of data storage, user interaction, and decision-making.

- **User Interaction & Interface:** Users can seamlessly interact with the system using text or voice commands via a Telegram Chatbot, which communicates with the core system using the HTTP protocol. To enhance user familiarity, a dedicated Dashboard is provided to display real-time statuses and analytical charts.
- **Decision Engine & Machine Learning:** Utilizing historical telemetry stored securely in MongoDB, this engine employs compact Machine Learning (ML) models to assist in autonomous decision-making and to generate the data visualizations presented on the dashboard interface.
- **Orchestration:** Acting as the central coordinator, it processes high-level ML decisions and user intents (parsed from the Telegram bot), translates them into specific hardware commands, and dispatches them down to the IoT layer via MQTT for immediate execution.

The interaction between these two layers creates a seamless feedback loop: Layer 1 provides the physical context and executes actions via MQTT, while Layer 2 (accessible via HTTP through the chatbot) provides the intelligence, persistent data storage, and actionable decisions.

User Interface & Interaction

To provide a comprehensive and intuitive user experience, H.E.R.A. features a modern web-based dashboard built with React. This unified frontend interface allows users to monitor real-time environmental telemetry, manually control actuators, and interact directly with the multi-agent AI pipeline in one centralized location

1.4 Goal of the Project

Primary Objectives:

- Develop a Scalable IoT Architecture: Build a system capable of handling concurrent connections from multiple sensor nodes with low latency.
- Implement Data-Driven Automation: Move beyond simple timers to behavior-based automation using machine learning algorithms.
- Optimize Energy Consumption: Provide visualization and forecasting tools that empower users to reduce their carbon footprint and electricity costs.
- Enhance Security with AI: Utilize computer vision and anomaly detection techniques to provide superior security coverage compared to traditional motion sensors.

Scope of the Project:

- IoT Core Infrastructure: Implementation of essential smart home management features including device provisioning (registration), state synchronization (on/off status), real-time command dispatching via MQTT, and role-based access control (RBAC) for family members.
- Data Pipeline Integration: Development of automated ETL (Extract, Transform, Load) pipelines to ingest high-frequency sensor telemetry, clean noise, and structure data for analytical processing.
- AI-Driven Insights: Leveraging machine learning models to transform raw data into intelligence:
  - Regression Models: Forecasting energy consumption trends for the upcoming month.
  - Classification Models: Distinguishing between human presence and pets to filter false security alarms.
  - Clustering Algorithms: Grouping similar user activity patterns to recommend personalized automation schedules.
- Visualization & Reporting: Deployment of interactive dashboards featuring heatmaps (for room occupancy) and time-series charts to visualize historical environmental data and real-time system states.
- Digital Twin Integration & Simulation-to-Reality (Sim2Real):
  - Virtual Environment Replication: Development of a 3D digital twin of the home environment to synchronize and visualize the real-time status of IoT devices and environmental sensors.

  - Scenario Simulation: Creating "What-if" scenarios, such as fire outbreaks, gas leaks, or security breaches, to validate the responsiveness and accuracy of the system's emergency protocols without risking physical assets.
  - Sim2Real Framework: Ensuring that the control logic and AI models developed in the simulation can be seamlessly transitioned to physical hardware with minimal reconfiguration, facilitating robust testing and reducing the risk of equipment failure during deployment.

- Hardware & Architecture Scope: To establish precise physical boundaries and fulfill the required minimum number of input/output (I/O) interfaces, the system's hardware is distributed across two primary computational nodes:
  - Input Sensors: Integrates a Temperature/Humidity Sensor and a Light Sensor to continuously capture multi-modal environmental telemetry.
  - Output Actuators & Displays: Utilizes an LCD screen for real-time state visualization, alongside physical actuators including LED lights, a Mini Fan, and a Relay module to execute environmental adjustments and simulate appliance control.

- The Central Processing Node (PC): Serving as the cognitive intelligence hub (Layer 2), a computer system integrates a Microphone (Sound Sensor) as its primary input. This node is strictly dedicated to intensive audio acquisition and Speech-to-Text (STT) processing , subsequently feeding the Natural Language Understanding (NLU) pipeline before dispatching execution commands to the Edge Node via MQTT.

1.5 Business Requirements

Table 1: Business Requirements of the H.E.R.A system

ID	Stakeholder	Requirement Description	Success Criteria
BR1	Residents	The system must enhance residents' daily quality of life, convenience, and health protection by creating an automated living environment that understands their preferences, routines, and contextual needs within a medium-sized house of approximately 100–150 square meters.	The system accurately processes natural language commands and autonomously adjusts household environmental conditions, such as lighting and ventilation, across relevant zones without requiring residents to manually operate separate device applications.
BR2	Residents	The system must help residents improve household energy awareness and safety by providing monitoring, alerts, and analytical insights related to environmental conditions, device states, and energy-related usage patterns.	The system provides a resident-facing dashboard and real-time notifications that allow residents to monitor household status, review alerts, understand energy-related trends, and respond to abnormal conditions in a timely manner.

1.6 Functional Requirements

Table 2: Functional Requirements of the H.E.R.A system

ID	Functional Requirement
FR1	The system shall continuously collect environmental telemetry data across the residential house, including temperature, humidity, light intensity, device states, timestamps, and related room or zone identifiers.
FR2	The system shall accept resident commands through both voice and text interfaces, with voice input converted into text through a Speech-to-Text process.
FR3	The system shall process unstructured natural language commands in order to identify resident intents, target devices, target zones, and relevant command parameters for execution.
FR4	The system shall autonomously adjust household environmental conditions such as lighting and ventilation based on resident commands, sensor conditions, contextual rules, or learned usage patterns.
FR5	The system shall execute control actions on connected household devices and actuators through the IoT control layer.
FR6	The system shall monitor abnormal conditions and generate real-time alerts for residents when unsafe events, environmental anomalies, or unusual device states are detected.
FR7	The system shall provide analytical functions for household energy-related monitoring, pattern identification, and forecasting to support resident awareness and decision-making.
FR8	The system shall apply classification or anomaly detection models to reduce false alerts and improve the reliability of resident-facing notifications.
FR9	The system shall provide an interactive resident-facing dashboard that visualizes real-time household status, zone-based telemetry, historical data, alerts, and analytical insights.
FR10	The system shall record resident commands, sensor readings, device states, alerts, and analytical outputs in a persistent activity log for review, troubleshooting, and future system improvement.

1.7 Non-Functional Requirements

Table 3: Non-Functional Requirements of the H.E.R.A system

ID	Non-Functional Requirement
NFR1	The system should provide a user-friendly and hands-free interaction experience for residents through intuitive natural language communication and clear feedback.
NFR2	The system should respond to resident commands and environmental events with low latency to ensure seamless household automation.
NFR3	The system should operate reliably and maintain stable sensing, communication, and control during normal operation in a medium-sized residential house of approximately 100–150 square meters.
NFR4	The system should achieve high accuracy in intent recognition, command interpretation, anomaly detection, and alert classification to avoid incorrect actions and minimize false notifications.
NFR5	The system must ensure security and privacy for resident data, telemetry data, and voice input, while protecting communication channels from unauthorized access.
NFR6	The architecture should be scalable so that additional rooms, zones, devices, sensors, and analytical modules can be integrated within the medium-sized residential house without major redesign.
NFR7	The system should be maintainable, modular, and easy to test, update, and troubleshoot over time.
NFR8	Essential system functions should remain available as much as possible, including local edge-level operation for critical sensing and control tasks when internet connectivity is unavailable.

1.8 Data Requirements

Table 4: Data Requirements of the H.E.R.A system

ID	Data Requirement
DR1	The system shall store time-series sensor telemetry data, including temperature, humidity, light intensity, timestamps, and related room, zone, device, or environment identifiers.
DR2	The system shall maintain current and historical records of household device states, command execution results, and actuator responses.
DR3	The system shall store resident interaction data, including command history, interaction sessions, detected intents, generated responses, and execution outcomes.
DR4	The system shall store alert events, anomaly scores, classification outputs, and severity levels for resident safety and environmental monitoring.
DR5	The system shall maintain master and configuration data for residents, devices, rooms, zones, household environments, and system settings.
DR6	The system shall preserve logging data for debugging, resident review, incident tracing, and performance evaluation.
DR7	The system shall maintain metadata of analytical or AI models and store their generated insights, including forecasts and anomaly detection results.
DR8	The collected data should be timestamped, validated, and structured consistently so that it can be reliably used for automation, visualization, and machine learning tasks.

In summary, the requirements analysis defines the resident-centered business objectives of the H.E.R.A system together with the corresponding functional, non-functional, and data requirements needed for implementation and evaluation within a medium-sized residential house.

2 Firmware Architecture and Organization

2.1 Introduction to OhStem's Yolo UNO and AIoT Kit

In this project, the hardware subsystem is centred around the Ohstem Yolo UNO development board, which serves as the main embedded controller of the H.E.R.A system. As the local control unit, it is responsible for acquiring sensor data, communicating with the upper software layer, and executing control commands for output devices such as the LCD, RGB LED, fan, and relay. Developed by Ohstem, the Yolo UNO is built on a customised ESP32-S3 platform and is intended to support rapid prototyping, embedded control, and IoT-oriented application development through its user-friendly and integration-ready design.

At the core of the Yolo UNO is the ESP32-S3, a high-performance dual-core microcontroller developed by Espressif Systems. Based on the Xtensa LX7 architecture, it features two 32-bit processors operating at up to 240 MHz, together with built-in Wi-Fi 802.11 b/g/n (2.4 GHz) and Bluetooth 5.0 Low Energy connectivity. These capabilities make it well suited for intelligent embedded systems that require reliable wireless communication and real-time interaction with external devices.

The specific version used in this project is the ESP32-S3-N16R8, which provides 16 MB of flash memory and 8 MB of PSRAM. This hardware configuration offers sufficient resources for firmware development, peripheral integration, wireless communication, and interaction with the CoreIoT platform for remote monitoring and device control.

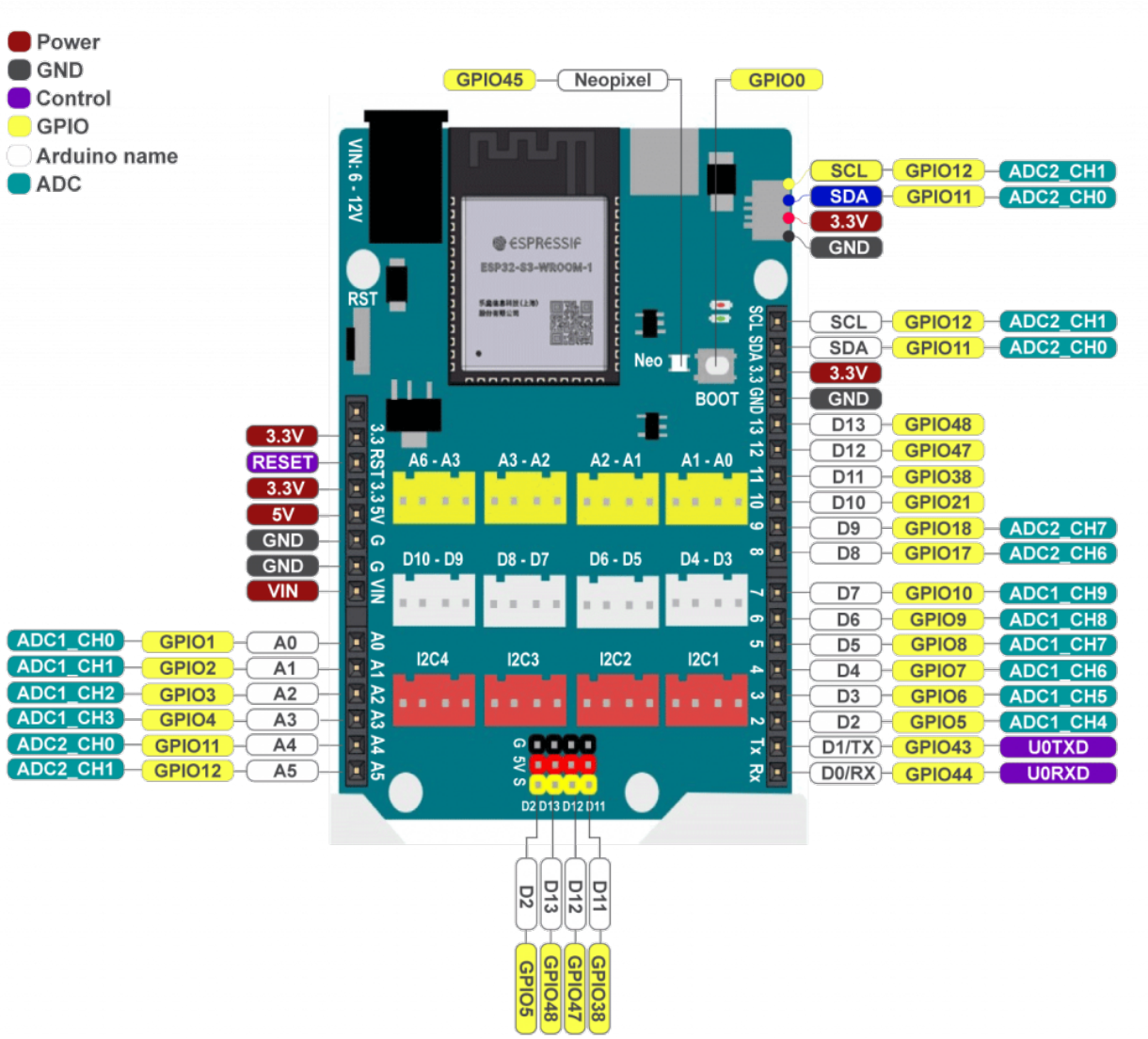

Figure 2.1: ESP32-S3-N16R8 based Ohstem Yolo UNO development board and pinout.

For firmware development and deployment, this project uses PlatformIO to program the ESP32-S3 microcontroller on the Yolo UNO board. PlatformIO is an open-source ecosystem for IoT development that supports multiple platforms and frameworks, providing a unified environment for building, uploading, testing, and maintaining embedded applications.

In addition to the controller board itself, many of the peripheral devices used in the system, including input modules, output modules, and environmental sensors, are selected from the OhStem AIoT Kit. This kit provides a convenient collection of plug-and-play modules, allowing the hardware components of the H.E.R.A system to be integrated more easily, safely, and consistently during development and experimentation.

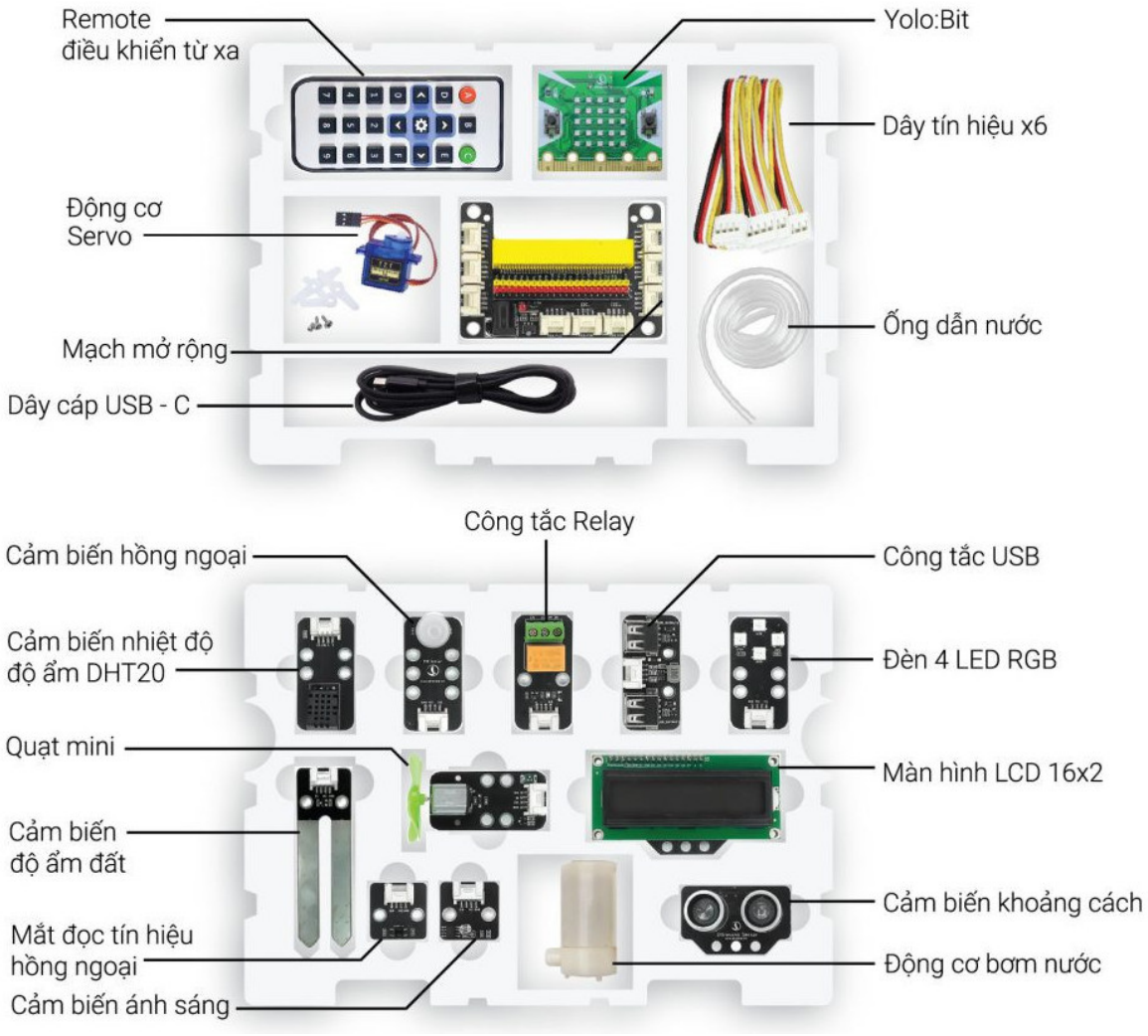

Figure 2.2: OhStem AIoT Kit and its main hardware modules.

Figure 2.2 shows the OhStem AIoT Kit, which provides a set of plug-and-play modules for IoT and embedded-system prototyping. The kit includes a variety of common input and output devices, such as the DHT20 sensor, infrared modules, light sensor, relay, RGB LED, LCD display, mini fan, ultrasonic sensor, and water pump.

In the H.E.R.A system, the AIoT Kit serves as the main source of peripheral modules for sensing and actuation. Its modular design simplifies hardware integration and supports faster, safer, and more convenient prototype development.

2.2 Hardware Device Inventory - Proof of Concept - PoC

Table 5: Main hardware devices used in the H.E.R.A system

No.	Device Name	Qty.	Primary Function
1	Central Processing Node (PC)	1	Performs high-level processing tasks, including Speech-to-Text (STT), Natural Language Understanding (NLU), and command coordination for the overall system.
2	Microphone / Sound Sensor	1	Captures user voice input for speech-based interaction and forwards audio data to the processing layer.
3	Yolo UNO Development Board	1	Serves as the main embedded controller, handling sensor data acquisition, MQTT communication, and actuator control.
4	Temperature and Humidity Sensor (DHT20)	1	Measures ambient temperature and humidity for environmental monitoring and automated control decisions.
5	Light Sensor (LDR)	1	Detects surrounding light intensity to support context-aware lighting automation.
6	LCD Display	1	Displays real-time sensor readings, device states, and general system status information.
7	RGB LED Module (WS2812)	1	Provides visual output for lighting control demonstration and system status indication.
8	Mini Fan	1	Simulates ventilation or cooling control in response to environmental conditions or user commands.
9	Relay Module	1	Enables switching control of external electrical loads or household appliances.

2.3 Functional Classification of Hardware Devices

To provide a clearer view of the hardware architecture, the devices used in the H.E.R.A system can be classified according to their functional roles. Rather than considering only input and output functions, this classification also includes the processing and control units that coordinate sensing, communication, and actuation within the system.

Table 6: Functional classification of hardware devices in the H.E.R.A system

Category	Devices	Role in the System
Processing and Control Units	Central Processing Node (PC), Yolo UNO Development Board	These units form the computational and control core of the system. The PC performs high-level processing tasks such as speech recognition, natural language understanding, and command orchestration, while the Yolo UNO executes embedded control tasks including sensor acquisition, MQTT communication, and actuator management.
Input Devices	DHT20 temperature and humidity sensor, LDR light sensor	These devices collect environmental data, specifically temperature, humidity, and ambient light intensity, then forward it to the controller for processing and decision-making.
Output Devices	LCD display, RGB LED module, mini fan, relay module	These devices present system information and perform physical responses, such as visual feedback, airflow generation, and switching external electrical loads or appliances.

In summary, the H.E.R.A platform combines sensing devices, embedded control hardware, and output actuators into an integrated smart-home architecture. This functional organisation enables the system not only to perceive environmental conditions and user input, but also to process commands intelligently and respond through appropriate physical actions.

2.4 Software Environment

The software environment and main libraries used in this project (which are already included) are:

- PlatformIO plug-in with the right toolchain.
- FreeRTOS as the real-time operating system.
- C/C++ as the main programming language for embedded tasks.
- Libraries for the DHT20 sensor, NeoPixel (Adafruit NeoPixel or equivalent), and I^2C LCD.
- Python 3.12.5 environment for data collection, cleaning, and model training.
- Telegram Bot API accessed from Python scripts.

2.5 Project Source Code Organization

2.5.1 Main Project Directory Tree

```
MP-AI-252/                      # Root workspace of the H.E.R.A project
|-- firmware/                   # ESP32/Yolo UNO firmware
|   |-- boards/                 # Custom PlatformIO board definition
|   |-- data/                   # Web/configuration assets stored on the board
|   |-- include/                # Firmware headers and shared declarations
|   |-- lib/                    # Local embedded libraries
|   |   |-- DHT20/              # Temperature and humidity sensor driver
|   |   |-- LCD/                # I2C LCD driver
|   |   `-- PubSubClient/       # MQTT client library
|   |-- src/                    # FreeRTOS tasks and device-control logic
|   `-- test/                   # Firmware test and validation workspace
|
|-- backend/                    # Backend, AI runtime, and data services
|   |-- Database/               # Express API and MongoDB integration
|   |-- HERA/                   # Multi-agent AI assistant runtime
|   |   |-- adapters/           # Telegram and voice input adapters
|   |   |-- agents/             # Specialist AI agents
|   |   |-- core/               # LLM, MQTT, event, and tool services
|   |   |-- domain/             # Smart-device catalog and execution logic
|   |   |-- memory/             # Session/profile memory backed by MongoDB
|   |   |-- orchestration/      # Agent routing graph and runtime state
|   |   |-- runtime/            # Tool execution, policy, and verification
|   |   |-- services/           # Higher-level assistant services
|   |   |-- telemetry/          # Telemetry schema and storage layer
|   |   `-- web_search/         # Web, news, and weather search utilities
|   |-- MQTT_Broker/            # Local MQTT broker manager and simulator
|   `-- Telegram Bot/           # Telegram helper utilities
|
|-- frontend/                   # Frontend workspace
|   `-- hera-dashboard/         # React/Vite monitoring dashboard
|       |-- public/             # Static assets served by the web app
|       `-- src/                # Dashboard source code
|           |-- assets/         # UI images and static resources
|           |-- components/     # Reusable React components
|           |-- pages/          # Dashboard pages
|           `-- services/       # API and MQTT client code
|
|-- docs/                       # Documentation and LaTeX report sources
|   |-- content/                # Report chapter sources
|   |-- image/                  # Figures and visual assets
|   |-- style/                  # LaTeX style and bibliography files
|   `-- upgrade/                # Technical notes and improvement plans
|
|-- evaluation/                 # Evaluation scripts and measured results
|-- setup.ps1                   # Development-environment setup helper
|-- requirements.txt            # Python dependency list used by setup.ps1
`-- README.md                   # Project overview and run instructions
```

2.5.2 Description of Main Directories and Files

The project repository is divided into several main source-code areas. Minor repository metadata and editor-related configuration files are omitted here so that the description focuses on the system components that directly implement H.E.R.A.

- **firmware/**: embedded firmware source tree for the ESP32-based Yolo UNO device. This module implements low-level sensor acquisition, actuator control, local device state management, Wi-Fi/MQTT communication, and the board configuration interface.

  - **boards/**: custom board definition for the Yolo UNO target used by the firmware build system.
  - **data/**: static files for the board-side web/configuration portal, including HTML, CSS, and JavaScript assets.
  - **include/**: C/C++ headers for shared firmware modules. Important headers include `global.h`, `main.h`, `mqtt_handle.h`, `analog_manager.h`, `digital_manager.h`, `board_config_server.h`, `button_handler.h`, `LCD_display.h`, `led_display.h`, `neo_display.h`, `sensor_dht20.h`, and `mq2_reader.h`.
  - **lib/**: embedded libraries used by the firmware:
    * **DHT20/**: driver implementation for the DHT20 temperature and humidity sensor.
    * **LCD/**: I^2C LCD driver used by the display module.
    * **PubSubClient/**: MQTT client library used for broker communication.
  - **src/**: main firmware implementation. The entry point `main.cpp` initializes semaphores, serial communication, I^2C, Wi-Fi configuration, and FreeRTOS tasks for buttons, digital/analog I/O, MQTT, DHT20 sensing, LCD output, LED output, and NeoPixel display.
  - **test/**: workspace for firmware experiments and validation.

- **backend/**: backend and AI service layer. This area contains the multi-agent assistant, MQTT infrastructure, database/API service, and Telegram utilities.
  - **Database/**: Node.js/Express API connected to MongoDB. It serves authentication, telemetry queries, latest sensor values, device/activity data, and model-setting endpoints for the dashboard.
  - **HERA/**: Python multi-agent assistant runtime. The main entry points are `main.py`, which builds the assistant runtime, and `api_server.py`, which exposes dashboard-facing HTTP endpoints while keeping device control on the same MQTT/tool-execution path as the assistant.
    * **adapters/**: adapters for external channels such as Telegram and voice input.
    * **agents/**: specialist agents for orchestration, device control, and web research.
    * **core/**: common services such as MQTT service, LLM service, event bus, message model, runtime settings, and tool registry.
    * **domain/**: device catalog and device-execution layer that maps assistant or dashboard commands to controllable smart-home targets.
    * **memory/**: session, profile, retrieval, and action-summary memory backed by MongoDB when available.
    * **orchestration/**: graph and state definitions used to route user requests through the assistant runtime.
    * **runtime/**: capability registry, policy engine, tool runner, read-tool runner, and verification service.
    * **services/**, **telemetry/**, and **web_search/**: higher-level services for anomaly analysis, telemetry reporting, response composition, telemetry storage, and external information retrieval.
  - **MQTT_Broker/**: local MQTT broker manager and simulator scripts. These utilities support both simulated operation and real-device operation by publishing telemetry, handling RPC topics, and optionally persisting telemetry to MongoDB.
  - **Telegram Bot/**: helper scripts for sending Telegram messages and testing notification flows.

- **frontend/**: frontend workspace for the user-facing dashboard.
  - **hera-dashboard/**: React/Vite dashboard for login, monitoring, analytics, floor-plan/device interaction, and model/runtime settings.
    * **public/**: static assets served directly by the web application, including icons and floor-plan resources.
    * **src/**: application source code, organized into:
      · **assets/**: static UI images and resources.
      · **components/**: reusable layout, dashboard, floor-plan, and chat components.
      · **pages/**: page-level views such as login, home, analytics, device/floor-plan, and settings screens.
      · **services/**: API and MQTT client code used for frontend–backend communication.

    * `App.jsx`, `main.jsx`, and `index.css` define the application shell, rendering entry point, and main styling layer.

- **docs/**: documentation and report-writing resources.
  - **content/**: LaTeX source files for the report chapters.
  - **image/**: figures, UI screenshots, diagrams, and assets used in the report.
  - **style/**: LaTeX style files, bibliography, cover, member list, and shared formatting definitions.
  - **upgrade/**: technical design notes, audits, and implementation plans created during system improvement.
- **evaluation/**: evaluation scripts and result files used to measure AI-agent behavior, database performance, firmware MQTT/RPC latency, firmware heap usage, and dashboard floor-plan performance.
- `setup.ps1`: Windows PowerShell setup wizard created by the development team to make the local development environment easier to prepare. The script checks for Git and Python 3.12, creates or reuses the project `.venv`, checks and installs Python packages from `requirements.txt`, installs `pre-commit`, sets up the Git hook when available, and supports dot-sourcing so the virtual environment can remain activated in the current terminal session.

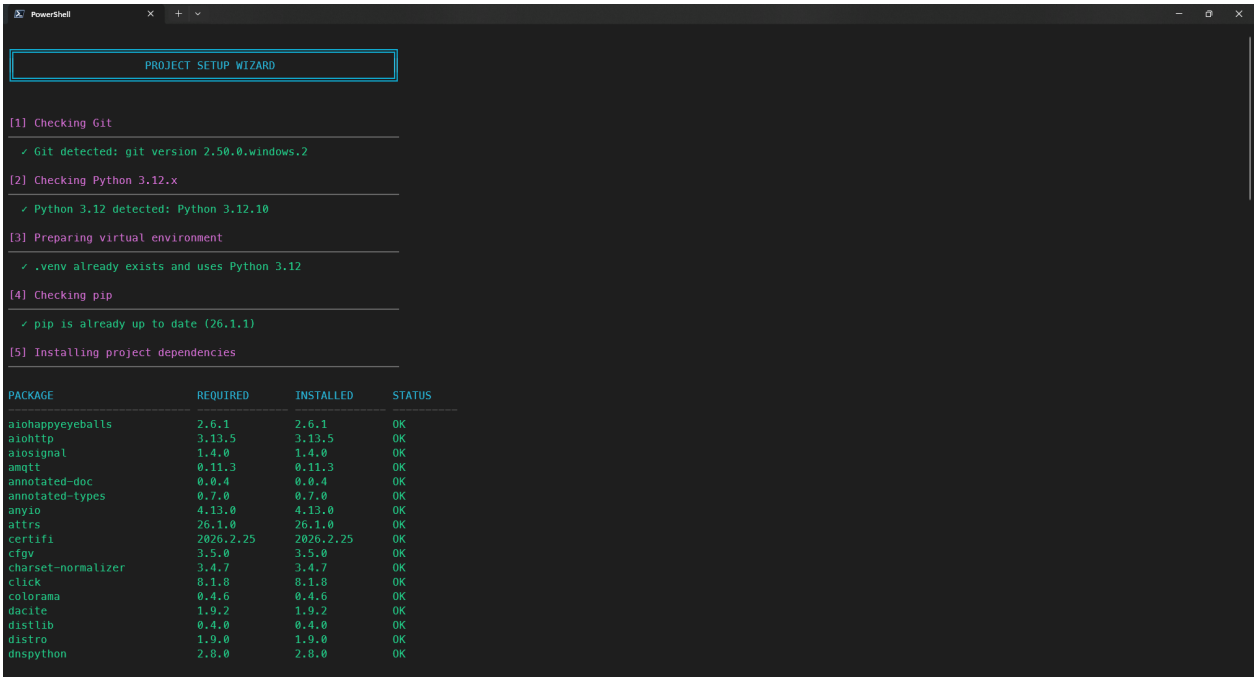

Figure 2.3: Initial checks performed by the `setup.ps1` development-environment setup script.

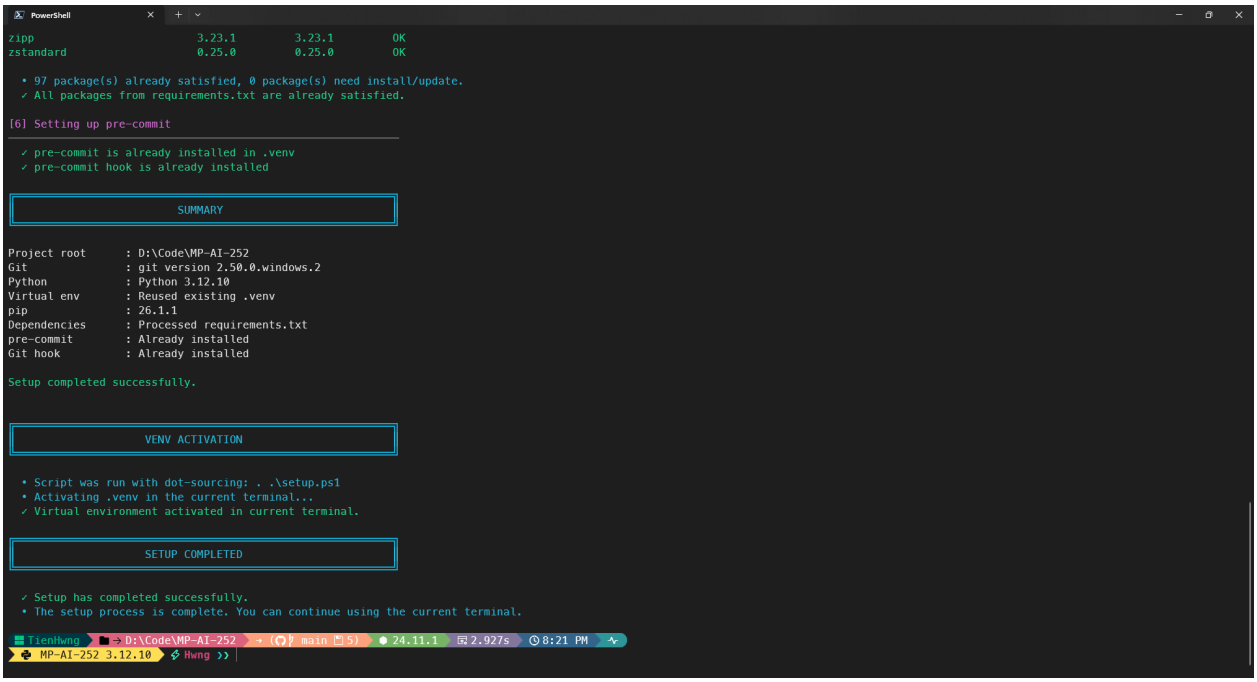

Figure 2.4: Dependency installation and environment preparation summary produced by `setup.ps1`.

- `README.md`: main project overview and run guide, including the major runtime entry points and service endpoints.

2.6 Configuration Macros and System Modes

An important design decision is the introduction of a centralized configuration header (included at the beginning of `global.h`) that defines all operational parameters using preprocessor macros. This header groups together:

- **Mode flags**: debug and logging options (e.g., enabling or disabling debug output and payload display).
- **Task delays and timeouts**: delays for LED blinking, NeoPixel updates, sensor reading, TinyML inference, CoreIOT publishing, and Wi-Fi/MQTT retry intervals.
- **GPIO mappings**: pins for the on-board LED, NeoPixel data line and LED count.
- **I²C pins**: SDA and SCL GPIO mapping for the LCD module and other I²C peripherals.

An excerpt of the configuration header is shown below:

```
// ----- General Settings -----
// Mode Settings
#define IS_DEBUG_MODE               false
#define IS_MONITOR_MODE             false
#define IS_SHOW_DHT20_STATUS        false
#define IS_SHOW_LED_STATUS          false
#define IS_SHOW_DIGITAL_STATUS      false
#define IS_SHOW_ANALOG_STATUS       false
#define IS_SHOW_BUTTON_STATUS       false
#define IS_SHOW_NEO_STATUS          false
#define IS_SHOW_LCD_STATUS          false
#define IS_SHOW_PAYLOAD             false
#define IS_SHOW_INFERENCE_RESULT    false

#define IS_SHOW_MQ2_STATUS          false
#define IS_SHOW_BOT_STATUS          false
#define IS_SHOW_SENSOR_LOG          false

// Time Delays (in milliseconds)
#define LED_BLINKY_DELAY_MS         1000
#define NEO_DISPLAY_DELAY_MS        1000
#define TEMP_HUMI_DELAY_MS          1000
#define MAIN_SERVER_DELAY_MS        1000
#define TINY_ML_DELAY_MS            5000
#define POLL_FROM_SERVER_DELAY_MS   1000
#define CORE_IOT_DELAY_MS           10000

#define LED_READER_DELAY_MS         1000
#define HUMID_READER_DELAY_MS       1000

#define LONG_PRESS_MS               3000
#define DEBOUNCE_MS                 50

#define ANALOG_READ_DELAY_MS        10
#define ANALOG_DEBOUNCE_MS          80

#define WIFI_CONNECT_TIMEOUT_MS     10000
#define WIFI_RETRY_INTERVAL_MS      5000
#define MQTT_RETRY_INTERVAL_MS      5000
#define SENSOR_LOG_DELAY_MS         2000
#define GAS_MONITOR_DELAY_MS        2000
// ----- END General Settings -----

// ----- Board default ports mapping -----
// Digital Grove Ports
#define DIGITAL_PORT_1_PIN          18      // D9
```

```
#define DIGITAL_PORT_2_PIN          10      // D7
#define DIGITAL_PORT_3_PIN          8       // D5
#define DIGITAL_PORT_4_PIN          6       // D3

#define DIGITAL_PORT_1_SUB_PIN      21      // D10
#define DIGITAL_PORT_2_SUB_PIN      17      // D8
#define DIGITAL_PORT_3_SUB_PIN      9       // D6
#define DIGITAL_PORT_4_SUB_PIN      7       // D4

// Analog Grove Ports
#define ANALOG_PORT_1_PIN           4       // A3
#define ANALOG_PORT_2_PIN           3       // A2
#define ANALOG_PORT_3_PIN           2       // A1
#define ANALOG_PORT_4_PIN           1       // A0

// I2C Pins
#define I2C_SDA_PIN                 11      // A4
#define I2C_SCL_PIN                 12      // A5
// ----- END Board default ports mapping -----

// ----- GPIO Pins Definitions -----
// Buttons
#define BOOT_PIN                    0
#define BUTTON_PIN                  47

// Grove's port devices
#define WS2812_PIN                  DIGITAL_PORT_1_PIN
#define IR_RECEIVE_PIN              DIGITAL_PORT_2_PIN
#define MINI_FAN_PIN                DIGITAL_PORT_3_PIN
#define RELAY_PIN                   DIGITAL_PORT_4_PIN

#define ANALOG_GPIO_PIN             ANALOG_PORT_1_PIN
#define LIGHT_SENSOR_PIN            ANALOG_PORT_2_PIN
#define MQ2_SENSOR_PIN              ANALOG_PORT_3_PIN
#define SOIL_MOISTURE_PIN           ANALOG_PORT_4_PIN

// GPIO ports
#define LED_PIN                     48

#define NEO_LED_PIN                 45
#define NEO_LED_NUMBER              8

#define WS2812_NUMBER               4
// ----- END GPIO Pins Definitions -----

// ----- Sensor & Device Constants -----
#define V_REF                      3.3f
// ----- END Sensor & Device Constants -----
```

Listing 1: Excerpt of configuration macros

At system startup, this header is compiled into all relevant modules, and the macros are used instead of hard-coded literals inside the application logic. This design allows users to adjust system behavior (timing, modes and pin mapping) by editing a single header file, without modifying the core task implementations. This centralized configuration mechanism is an explicit extended feature beyond the original assignment.

3 Multi-Agent AI Pipeline Architecture

This section presents the conversational intelligence architecture of H.E.R.A. The current implementation is not a single chatbot placed in front of the IoT system. Instead, it is a graph-based multi-agent runtime that separates routing, device planning, telemetry reporting, anomaly analysis, web research, memory handling, policy checking, and final response composition. This separation is important because H.E.R.A must both communicate naturally with users and safely control physical devices. A natural-language error in a casual response is tolerable; an incorrect actuator command is not. Therefore, the architecture is designed around explicit contracts, bounded tool capabilities, deterministic guards, and auditable execution results.

3.1 Design Gap and Architectural Motivation

The core technical gap addressed by this architecture is the mismatch between open-ended language generation and safety-critical IoT control. Large language models are effective at interpreting natural language, but they are probabilistic systems. If an LLM is allowed to directly generate MQTT commands or directly decide whether an environmental condition is dangerous, the system becomes difficult to verify. In a smart-home context, this creates three risks:

- **Tool ambiguity:** user phrases such as “turn it off” or “make the room darker” require grounding to a concrete device, property, and value.
- **State inconsistency:** an LLM may claim a device was controlled even when the hardware is offline or already in the requested state.
- **Evaluation difficulty:** if routing, reasoning, tool execution, and response generation are mixed in one prompt, failures cannot be attributed to a specific layer.

H.E.R.A addresses this gap using an orchestrator-specialist architecture implemented as a LangGraph state machine. The LLM is used where semantic interpretation is valuable, while deterministic software is used for policy, validation, telemetry freshness, anomaly classification, and execution verification. This division is the central design principle of the AI subsystem.

3.2 Current H.E.R.A AI Architecture

Figure 3.1 shows the current conversational intelligence core of H.E.R.A. This diagram focuses only on the AI layer, not the full hardware, database, or frontend architecture of the whole system. The user request first enters the request-understanding stage, where the input is normalized and prepared as a structured message. It is then passed to the H.E.R.A orchestrator, which coordinates routing, memory access, specialist selection, runtime tool execution, and final response generation. Supporting systems such as memory, specialist modules, safety verification, and the tool system are shown as indirect dependencies because they do not replace the main request path; instead, they provide context, capabilities, and validation to the core pipeline.

The high-level flow can be read from left to right. A natural-language request becomes a normalized request, the orchestrator decides how it should be handled, a specialist produces either a domain report or a tool-oriented plan, the runtime tool boundary validates and executes the required tool calls, and the response generator converts the verified result into a user-facing answer. This separation is important because it keeps language interpretation, tool execution, and final explanation as distinct stages. In particular, the LLM does not directly publish MQTT commands. It can only propose structured actions that must pass through the runtime boundary.

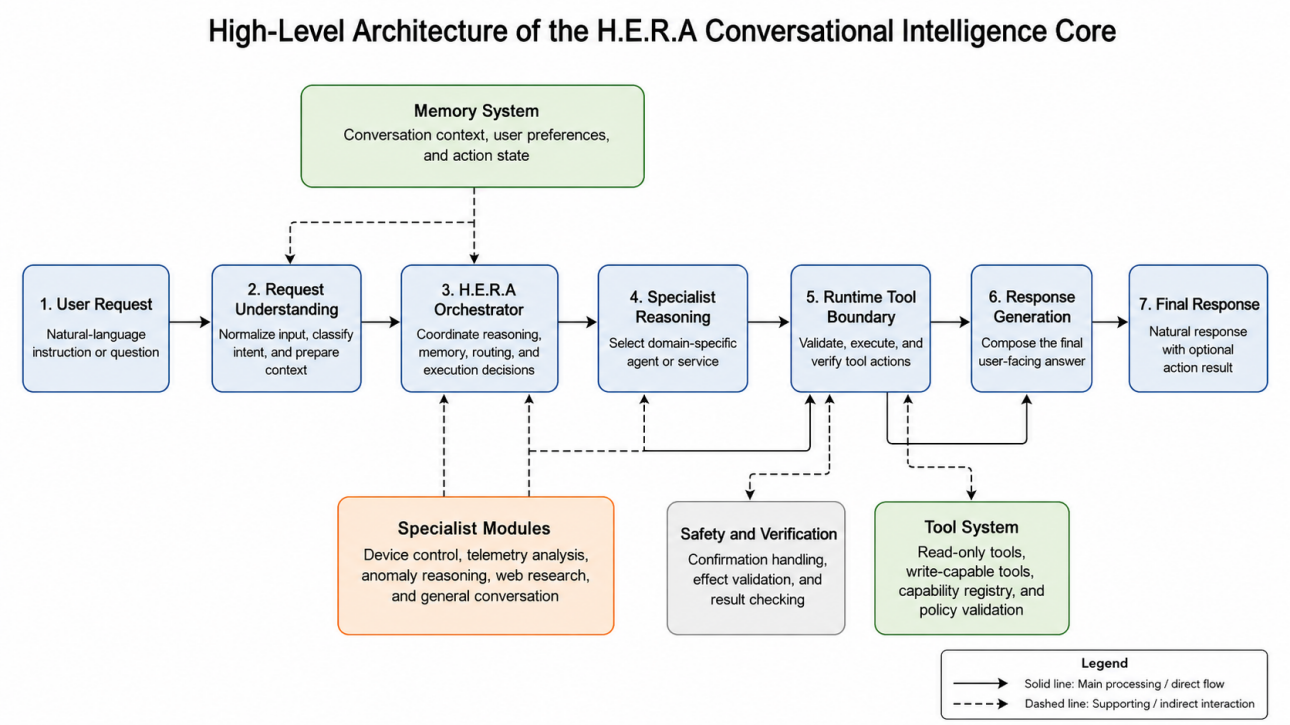

Figure 3.1: Current layered architecture of the H.E.R.A multi-agent AI subsystem.

Figure 3.2 expands the orchestration block shown in the high-level architecture. The orchestrator acts as the decision center between the user-facing API and the specialist/runtime layers. It first performs request-state intake, routes the message by intent, checks pending confirmation state, and selects the memory context required for the current request. The routing result then determines whether the request can be handled as general conversation, dispatched to a specialist, or sent into the tool-calling path. Only after this state gate does the system enter the runtime execution block. The final response is composed from the specialist report and any verified runtime result, rather than from unconfirmed model text alone.

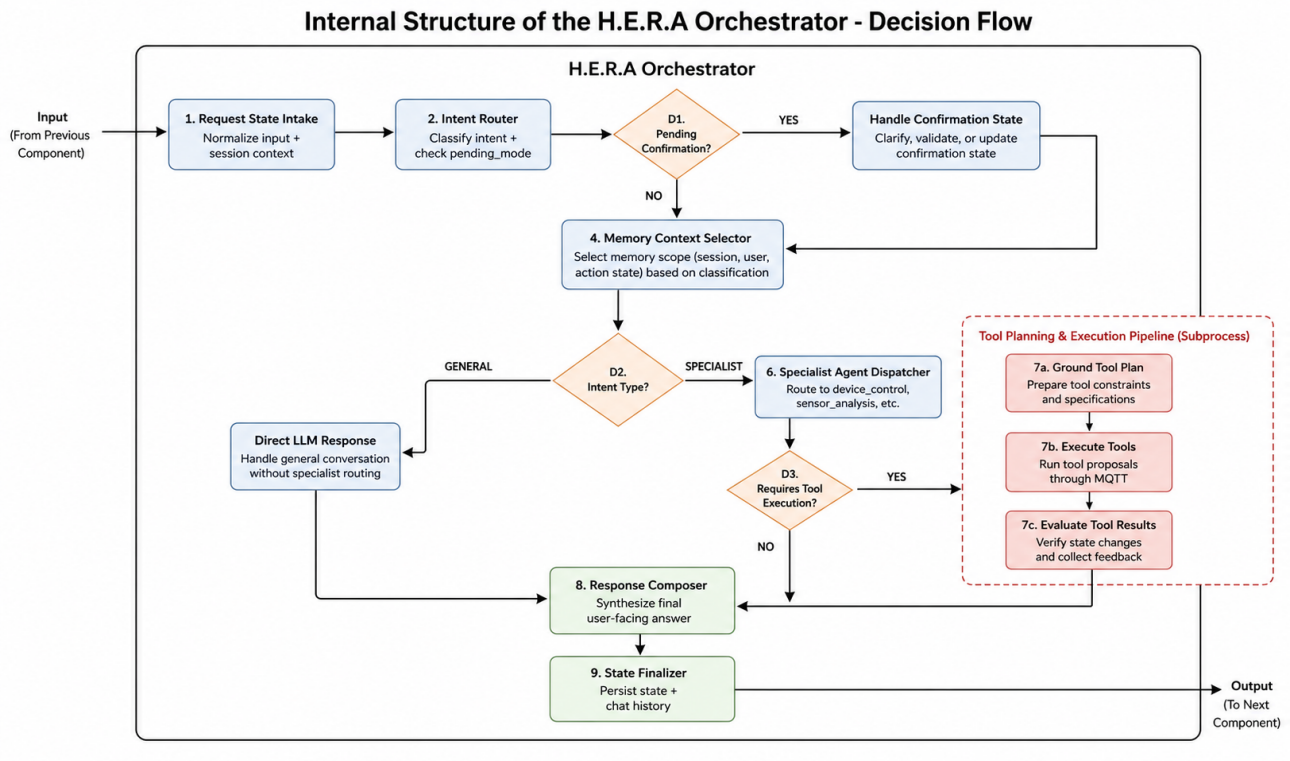

Figure 3.2: Detailed orchestrator pipeline block inside the H.E.R.A multi-agent AI subsystem.

The red subprocess in Figure 3.2 represents the tool-calling and runtime execution block. This block is expanded in Figure 3.3. Its input is a set of grounded tool calls or capability proposals produced by the specialist path. Its output is a normalized tool-execution result that returns to the response composer. Between those two points, the runtime performs capability validation, policy checking, read/write routing, optional confirmation handling, tool execution, result normalization, and effect verification.

The read-only path is used for operations such as retrieving current telemetry, device status, or recent telemetry windows. These operations do not mutate external device state and can therefore be executed through a simpler read-tool runner. The write-capable path is stricter. It checks whether the target and arguments are valid, whether policy allows the action, whether user confirmation is required, and whether the physical effect can be verified after execution. This design keeps tool calling auditable: every runtime result has an explicit status, policy decision, and verification fact that can be used by the response composer and later evaluation logs.

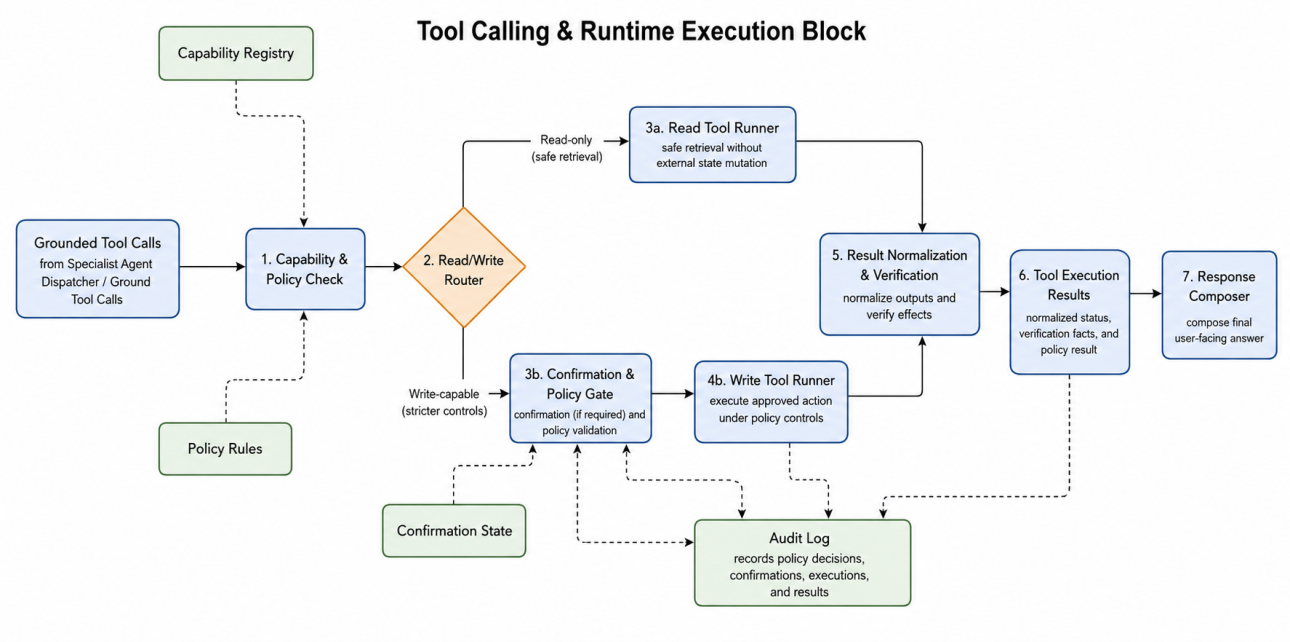

Figure 3.3: Internal flow of the H.E.R.A tool-calling and runtime execution block.

Together, Figures 3.1, 3.2, and 3.3 describe the same request flow at three levels of detail. The first figure shows where the AI core sits conceptually, the second figure shows how the orchestrator chooses the processing route, and the third figure shows how concrete tool calls are validated and executed. This layered presentation reflects the implementation principle of H.E.R.A: the AI system may interpret and plan, but physical effects must pass through deterministic runtime boundaries before they are reported back to the user.
The architecture contains five main functional layers:

Table 7: Functional layers of the H.E.R.A AI subsystem

Layer	Responsibility
Interface layer	Receives user input from REST/dashboard or other adapters and converts it into a unified `UserMessage`.
Orchestration layer	Classifies intent, selects memory scope, handles pending confirmations, routes to specialists, and composes the final user-facing response.
Specialist layer	Performs domain-specific planning or reporting: device control, telemetry reporting, anomaly analysis, and web research.
Runtime layer	Converts plans into capability proposals, checks policy, executes read/write tools, verifies effects, and records structured results.
Infrastructure layer	Provides LLM inference, MQTT communication, MongoDB memory/telemetry storage, and external search services.

3.3 LangGraph Execution Model

The current H.E.R.A orchestrator is implemented as a LangGraph state machine rather than a simple sequential function. The graph is compiled with an in-memory checkpointer keyed by the user session, allowing the runtime to preserve thread state such as chat history, active device focus, pending confirmation, pending device clarification, and previous tool results. Figure 3.4 summarizes the graph.

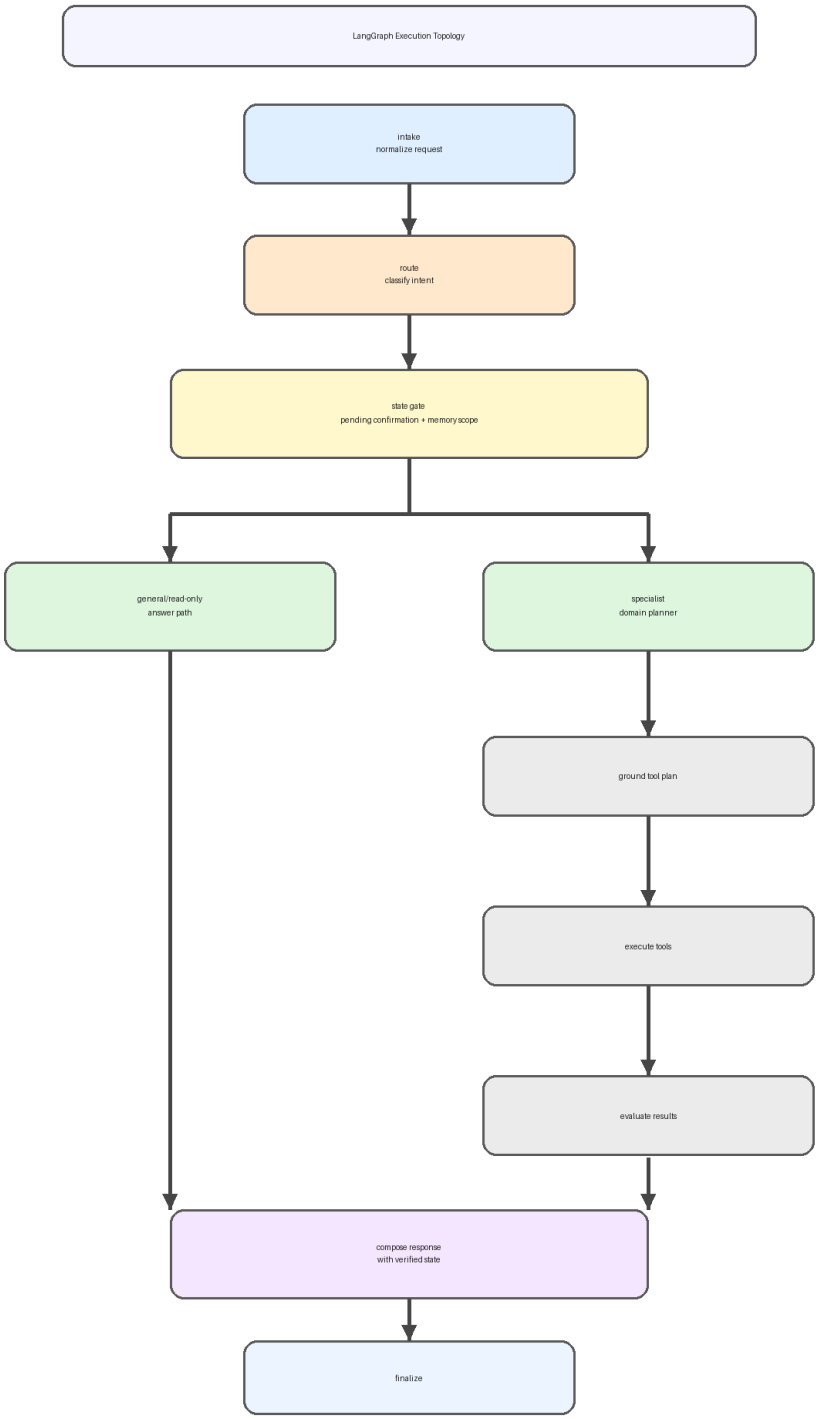

Figure 3.4: LangGraph execution topology of the H.E.R.A orchestrator.

The graph separates planning from execution. A specialist does not directly control hardware. It returns a structured report or a set of tool proposals. The runtime tool node then executes those proposals through the central tool runner. This allows policy enforcement and verification to be applied uniformly, whether the request originates from the chat interface or the dashboard.

3.4 Intent Taxonomy and Routing

The orchestrator recognizes five intent categories:

Table 8: Intent taxonomy used by the orchestrator

Intent	Specialist	Purpose
`device_control`	`device_control`	Parse actuator requests, generate capability proposals, and route write operations through policy and verification.
`sensor_query`	`telemetry_report`	Report current temperature, humidity, light, anomaly score, device state, and network state.
`anomaly_query`	`anomaly_analyzer`	Explain abnormal telemetry based on freshness, thresholds, anomaly score, and recent telemetry windows.
`web_search`	`web_research`	Retrieve current external information using bounded search/fetch services.
`general`	`orchestrator`	Handle greetings, help requests, and non-effectful conversation.

The routing prompt returns not only an intent but also a memory policy. The memory scope can be `none`, `session`, `actions`, `profile`, or `all`. This distinction prevents unnecessary memory retrieval for simple current-state questions while allowing contextual commands such as "turn it off" to use recent actions or active focus. The router also has a safety correction step: if a message is classified as a sensor or general request but the device planner semantically recognizes a valid actuator command, the orchestrator forces the full device-control graph.

3.5 Short-Path Optimization for Read-Only Requests

H.E.R.A includes an optimized short path for simple non-effectful requests. If the intent is `sensor_query`, `anomaly_query`, or trusted `general`, and the router declares that no memory is needed, the orchestrator can respond without executing the full graph. This is shown in Figure 3.5.

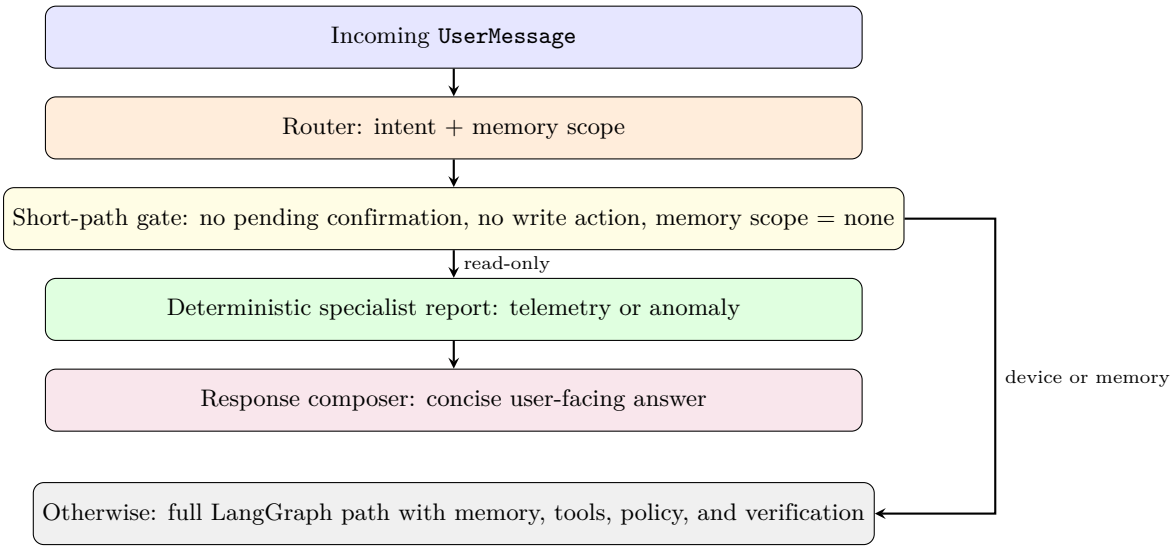

Figure 3.5: Short-path optimization for read-only AI requests.

This design reduces latency and cost because routine status questions do not require a full agent graph or a tool-execution cycle. More importantly, it preserves safety because device-control requests are explicitly excluded from the short path.

3.6 Specialist Components

3.6.1 Device Control Agent

The Device Control Agent converts natural language actuator requests into structured capability proposals. It uses the configured device-control model to parse a command into an action, target, optional property, optional value, optional condition, and confidence score. Supported actions include:

- `turn_on` and `turn_off` for binary device control.
- `status` for reading current device state.
- `set_device_value` for brightness, color, and fan-speed control.
- `set_sensor_value` for simulator-only sensor overrides.

The agent grounds natural language targets using a physical placement catalog. This allows the user to speak in room-level terms rather than internal device identifiers. Table 9 shows the current mapping between runtime device targets, physical locations, and user-facing names.

Table 9: Device placement mapping used by the H.E.R.A AI and digital-twin layers

Runtime target	User-facing device	Room	Supported control semantics
`main_led`	Living room light	Living Room	On/off control for the main indicator LED.
`neo_led`	Bedroom light	Bedroom	On/off control and brightness adjustment for the NeoPixel strip.
`ws2812`	Toilet light	Toilet	On/off control, brightness adjustment, and RGB color control.
`mini_fan`	Living room fan	Living Room	On/off control and fan-speed adjustment.
`relay`	Living room TV	Living Room	On/off relay control for the TV load.
`all_lights`	All lights	Multiple rooms	Group target for all lighting devices.
`all_devices`	All devices	Whole house	Broad group target; requires confirmation before execution.

The agent also supports conditional commands. For example, a request such as “turn on the fan if the temperature is above 35 degrees” is parsed as a device action plus a sensor threshold. The condition is then grounded against the current telemetry snapshot or a telemetry window before any write tool is produced. This prevents the model from executing a conditional command without validating the condition against data.

3.6.2 Telemetry Report Service

The Telemetry Report Service is deterministic. It does not ask an LLM to infer current sensor values. Instead, it reads the current MQTT snapshot through the read-tool boundary and returns a structured specialist report. The report includes the current snapshot, data-source metadata, availability flags, and reference thresholds for temperature, humidity, and anomaly score. The final response may be natural-language text, but the facts come from runtime telemetry.

3.6.3 Anomaly Analyzer Service

The Anomaly Analyzer Service is also deterministic. It first checks telemetry freshness using the configured stale threshold. If telemetry is stale or missing, the service reports an unknown current-state severity rather than making an unsupported conclusion. If telemetry is fresh, it classifies anomaly status using rule-based checks over temperature, humidity, and anomaly score. It also reads a recent telemetry window from MongoDB when available. The LLM is used only to explain the structured result, not to decide whether an anomaly exists.

3.6.4 Web Research Service

The Web Research Service handles external and time-sensitive information. It supports generic DuckDuckGo search, direct URL fetching, and specialized weather/news paths when the corresponding API keys are configured. Search results are trimmed before entering the final response composer, which bounds prompt size and reduces the risk of ungrounded answers. This specialist is separated from device control so external web content cannot directly trigger actuator commands.

3.7 Runtime Capability Model

The runtime layer exposes a small set of declared capabilities instead of allowing arbitrary generated code or arbitrary MQTT payloads. Table 10 summarizes the implemented capability registry.

Table 10: Runtime capabilities exposed to the H.E.R.A AI layer

Capability	Effect	Risk	Description
`get_device_status`	read	low	Reads current state of main LED, NeoPixel LED, WS2812 LED, relay, and mini fan.
`get_current_telemetry`	read	low	Reads current temperature, humidity, light, anomaly score, device states, and network state.
`get_telemetry_window`	read	low	Reads a recent telemetry summary window for threshold checks and anomaly diagnostics.
`turn_on_device`	write	medium	Turns on a selected device or group if the requested state differs from the current state.
`turn_off_device`	write	medium	Turns off a selected device or group if the requested state differs from the current state.
`set_device_value`	write	medium	Sets supported adjustable values such as LED brightness, WS2812 color, or fan speed.
`set_sensor_value`	write	medium	Overrides simulator sensor values for test and demonstration mode.

This capability design is intentionally parameterized. Instead of defining one function for each individual device action, the model produces a capability name and structured arguments. The benefit is twofold. First, the model chooses among fewer tool types, reducing selection ambiguity. Second, new devices can be added by extending the target catalog and validation rules instead of rewriting the orchestration logic.

3.8 Policy, Verification, and Safety

The policy engine evaluates every write proposal before execution. Read-only tools are allowed by default, but write tools are checked against runtime state and target validity. The implemented policies include:

- reject or ask for clarification when the target is missing or invalid;
- deny execution when the device reports MQTT offline;
- require confirmation before controlling `all_devices`;
- return `noop` when the requested state or value is already satisfied;
- coerce and validate brightness, fan speed, color, and simulator sensor values before execution.

After execution, the tool runner performs readback verification. For state changes, it polls the device status snapshot until the expected state is observed or the verification timeout expires. For value changes, it checks the appropriate value field. The final response is then grounded in execution results such as changed entities, unchanged entities, failed entities, policy reason, and verification status. This prevents the assistant from claiming success solely because the LLM generated a tool call.

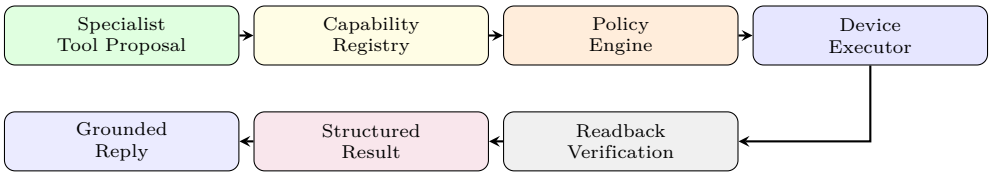

Figure 3.6: Safe tool execution path for effectful device-control requests.

3.9 Memory and Discourse Grounding

The graph maintains session state through a LangGraph checkpointer and optionally persists memory through MongoDB. H.E.R.A uses memory selectively rather than attaching all history to every request. This design avoids unnecessary context growth and reduces the chance that old conversation turns influence unrelated device commands. The system uses three practical memory concepts:

- **Chat history** for coherent multi-turn conversation.
- **Recent actions** for resolving follow-up commands such as "turn it off".
- **Active focus** for carrying a currently discussed device target across turns.

The router chooses whether memory is needed. If memory scope is `none`, read-only requests can use the short path. If the request depends on prior context, the graph retrieves memory before selecting the specialist. This gives H.E.R.A contextual ability without making every request expensive or unpredictable.

3.10 Model Provider and Current Configuration

The LLM layer is provider-agnostic through LiteLLM. H.E.R.A currently supports two providers:

- **Ollama:** local inference through `http://localhost:11434`, suitable for offline and privacy-sensitive operation.
- **OpenRouter:** cloud inference through the configured OpenRouter API base URL, suitable for flexible model access and easier deployment on machines without sufficient local GPU capacity.

The current runtime provider is `openrouter`. Model settings are loaded from MongoDB collection `model_settings` using document id `hera_model_settings`; if that document is unavailable, H.E.R.A falls back to code-level defaults. In the current project run, the dashboard settings API reports the model assignments shown in Table 11. The project environment file configures provider endpoints such as the Ollama API base URL and OpenRouter base URL, while model names are controlled by the runtime settings store.

Table 11: Current configured model assignment in the H.E.R.A runtime

Role	Ollama configuration	OpenRouter configuration
Orchestrator router/composer	`gemma4:e4b`	`nvidia/nemotron-nano-9b-v2`
Device Control Agent	`TechyShishy/ministral-3:3b`	`qwen/qwen3-30b-a3b-instruct-2507`
Sensor Analysis role	`TechyShishy/ministral-3:3b`	`nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free`
Anomaly Expert role	`TechyShishy/ministral-3:3b`	`nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free`

Although the settings schema still contains sensor and anomaly model fields, the current telemetry and anomaly specialists are deterministic services. Their model fields remain useful for future LLM-based explanation upgrades, but the present implementation deliberately keeps core telemetry and anomaly classification outside the LLM.

3.11 Model Selection Rationale

The model configuration follows a cost-capability split. Routing benefits from a smaller model because the output space is bounded and latency matters. Device control uses a stronger instruction-following model because it must parse multilingual commands, room-level aliases, device values, conditional clauses, and clarification cases. The local Ollama configuration prioritizes offline availability with compact models, while the OpenRouter configuration assigns a larger Qwen model to device planning and Nemotron-family models to routing and explanatory roles. This split reflects the practical difference between low-cost local operation and cloud-assisted capability.
The OpenRouter-side selection was not based only on model popularity. H.E.R.A was tested with simple dashboard prompts before integration, and the NVIDIA response behavior was the most stable among the candidate responses observed in the local comparison. Figures 3.7 and 3.8 show these prompt-level checks. These checks are not treated as a formal benchmark; rather, they provide project-specific evidence that the selected model family follows the expected conversational style before being placed into a constrained orchestrator runtime.

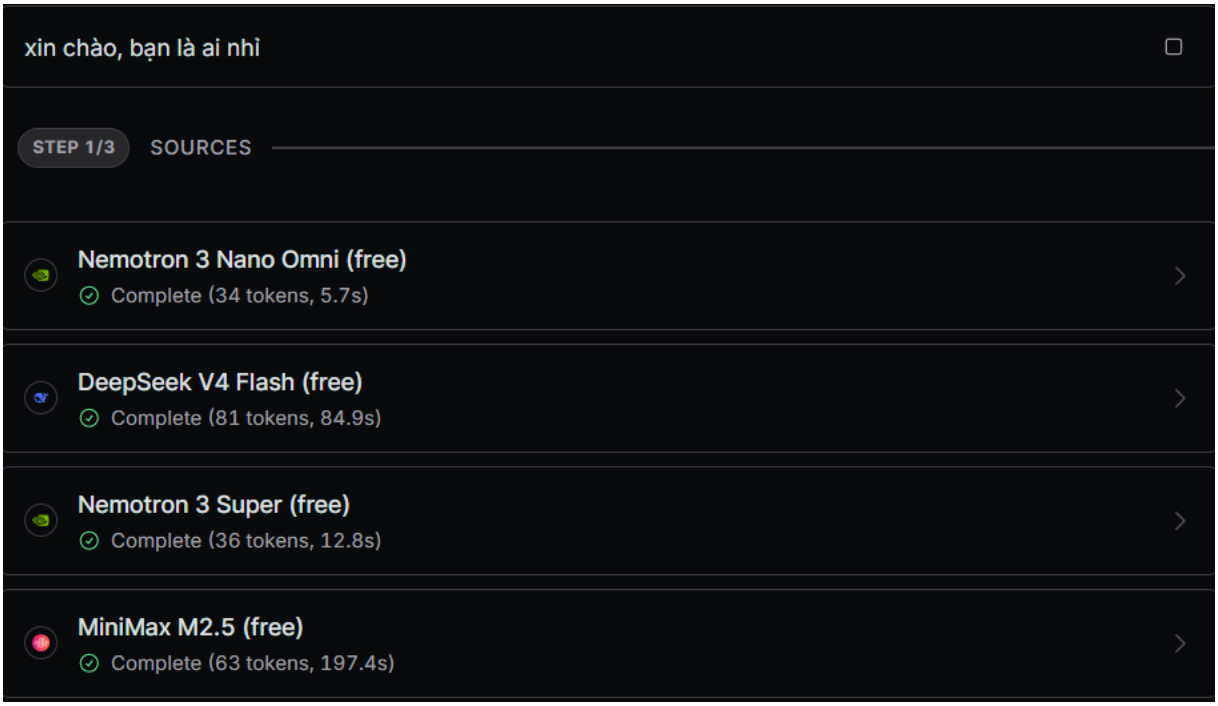

Figure 3.7: Initial prompt-level comparison used during model selection.

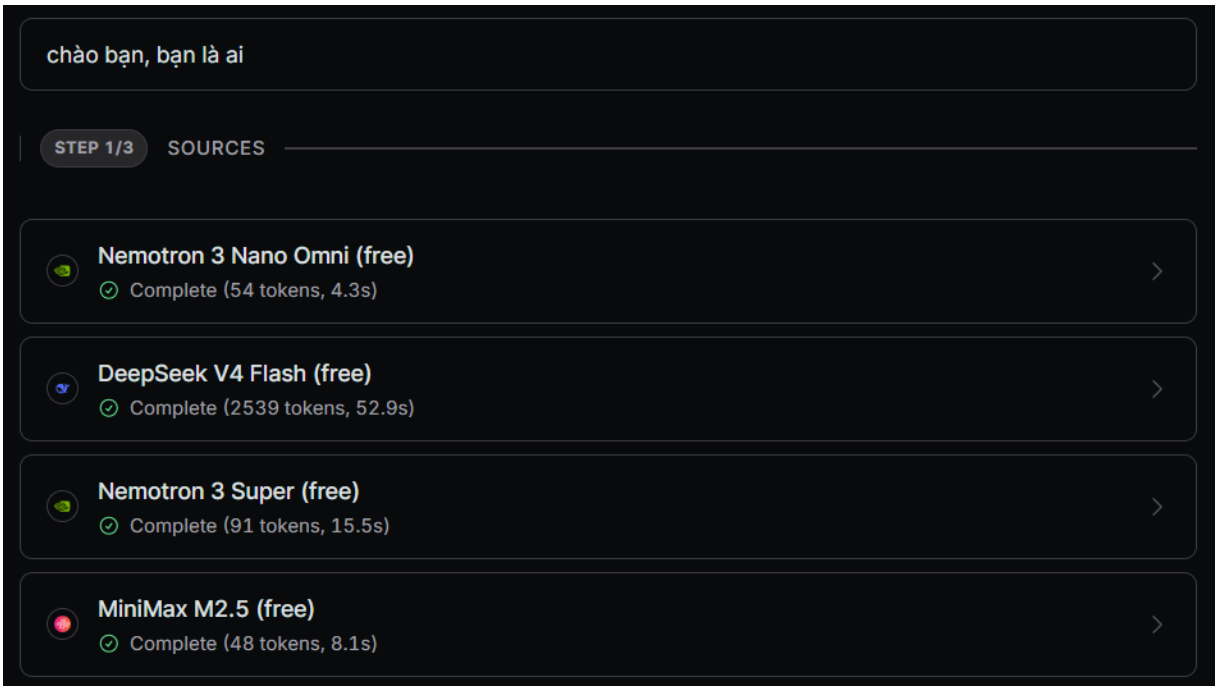

Figure 3.8: Second prompt-level comparison used to validate response stability.

Because the H.E.R.A workload is specific to language understanding, intent routing, and tool-oriented execution, the model choice was also checked against public benchmark evidence. NVIDIA's model card for `NVIDIA-Nemotron-Nano-9B-v2` reports stronger results than Qwen3-8B on instruction following, coding/reasoning, long-context evaluation, and Berkeley Function Calling Leaderboard v3 (BFCL v3), a benchmark designed to evaluate function/tool-calling capability.[1][2] The associated NVIDIA technical report further motivates the Nano-v2 family as an efficient hybrid Mamba-Transformer reasoning model, reporting comparable or better accuracy than similarly sized models with significantly higher inference throughput in long reasoning settings.[3] This external evidence matches the system-level requirement: the orchestrator model does not need to be the largest available model, but it must reliably follow instructions, preserve structured intent, and support agent/tool workflows.

[1] https://huggingface.co/nvidia/NVIDIA-Nemotron-Nano-9B-v2
[2] https://sky.cs.berkeley.edu/project/berkeley-function-calling-leaderboard/
[3] https://arxiv.org/abs/2508.14444

Table 12: External evidence used to justify the NVIDIA/Nemotron model family

Evidence Source	Reported Metric	Reported Result	Relevance to H.E.R.A
NVIDIA model card	IFEval instruction strict	90.3% for Nemotron Nano 9B v2 vs. 89.4% for Qwen3-8B	Supports the orchestrator role, where the model must follow routing and formatting instructions.
NVIDIA model card	BFCL v3 tool calling	66.9% for Nemotron Nano 9B v2 vs. 66.3% for Qwen3-8B	Directly relevant to tool-oriented agents because BFCL evaluates function/API calling accuracy.
NVIDIA model card	RULER 128K long-context	78.9% for Nemotron Nano 9B v2 vs. 74.1% for Qwen3-8B	Useful for multi-turn sessions where routing may depend on chat history, active focus, and recent actions.
NVIDIA technical report	Efficiency claim	Up to 6× higher inference throughput in long reasoning settings compared with similarly sized models	Relevant to a dashboard assistant because lower perceived latency matters during repeated chat interactions.
Project prompt checks	Simple user-facing prompts	NVIDIA responses were selected as the most stable in Figures 3.7 and 3.8	Confirms that the public benchmark fit also appears in the project-specific conversational style.

Table 13: Model role requirements for H.E.R.A

Role	Main requirement	Reason for model choice
Router	Structured intent classification	Small bounded output space; lower latency is more valuable than broad reasoning.
Device planner	JSON command parsing and tool grounding	Requires stronger instruction following and robust handling of device aliases, values, and conditions.
General response composer	Concise natural language	Needs fluent bilingual response generation but no direct tool access.
Telemetry/anomaly services	Deterministic computation	Implemented without LLM decision-making to keep safety and evaluation auditable.
Web research composer	Grounded summarization	Uses retrieved evidence but remains separated from actuator execution.

3.12 Dashboard Streaming Mode

The AI path described above still contains several latency sources: routing, model inference, specialist execution, optional tool execution, verification, and final response composition. Even when the backend completes correctly, returning the final answer as a single JSON payload makes the dashboard feel blocked until the full response is available. To reduce perceived latency, H.E.R.A introduces a streaming mode for the dashboard chat interface. This mode does not change the orchestrator's reasoning contract or the runtime tool boundary; it changes only how the already-generated assistant response is delivered to the frontend.

Streaming is enabled by adding the query parameter `stream=true` to the dashboard assistant endpoint. The backend endpoint `/api/assistant/message` first builds the same `UserMessage` object used by the non-streaming path and still calls the same orchestrator. After the orchestrator returns a structured response, the API server switches from a normal JSON response to a Server-Sent Events (SSE) stream. The SSE response uses `text/event-stream`, disables caching, keeps the connection open, and includes CORS headers so that the browser can consume the stream from the dashboard origin.

The backend streaming handler separates optional thinking metadata from final answer content. Thinking text, when present, is emitted in small three-character chunks with a short delay. Final answer text is emitted one character at a

time with an approximately 20 ms delay, which produces a readable character-by-character response in the browser. Each message follows an OpenAI-compatible delta shape, such as `{"choices":[{"delta":{"content":"H"}}]}`, and the stream terminates with `data: [DONE]`. If the orchestrator fails before streaming begins, the backend returns the same error information through an SSE error payload so the frontend can surface the failure consistently. On the frontend, `sendAssistantMessageStreaming()` in the dashboard API service sends the request with `stream=true`, attaches an `AbortSignal`, reads `response.body.getReader()`, decodes complete SSE lines, and accumulates content and thinking streams separately. The chat component creates an empty assistant placeholder before the first chunk arrives, tracks the active streaming message id with a React ref, and updates only that message as chunks arrive. This design avoids re-rendering unrelated chat history on every token. Where available, `ReactDOM.flushSync()` is used to force immediate rendering per chunk, preventing React batching from collapsing many chunk updates into one visual update.

Table 14: Streaming-mode design decisions in the H.E.R.A dashboard chat

Layer	**Implementation Decision**	**Design Rationale**
Backend API	The `/api/assistant/message` endpoint selects SSE delivery when `stream=true` after normal orchestrator execution	Preserves the existing AI control flow while changing only the delivery mode.
Transport	SSE chunks use `data: ...` lines and terminate with `[DONE]`	Simple browser-native unidirectional streaming fits assistant responses without requiring WebSocket state.
Chunking	Thinking chunks use three characters; answer chunks use one character with a 20 ms delay	Makes response progress visible and readable instead of appearing as a buffered block.
Frontend parser	`ReadableStream` reader parses SSE lines and accumulates full text	Supports progressive rendering while still retaining the complete final answer for chat history.
React state	Active message id is stored in a ref and updated directly through an indexed message update	Reduces per-chunk state work and avoids unnecessary updates to previous messages.
User control	`AbortController` backs the pause action	Closing the fetch stream stops frontend updates and prevents wasted backend/network work.
Fallback	If streaming returns JSON or fails, the frontend falls back to the non-streaming API path	Keeps the dashboard usable even when streaming is unsupported or interrupted.

This streaming design is therefore a presentation and responsiveness optimization, not a shortcut around safety. Tool execution still completes inside the runtime boundary before the final answer is streamed. The user receives the answer progressively, but the content remains grounded in the same verified orchestrator result as the non-streaming mode. This keeps the system design consistent with the rest of the H.E.R.A AI architecture: semantic reasoning and physical action remain controlled by the orchestrator and runtime layers, while the dashboard optimizes the delivery of the final user-facing response.

3.13 End-to-End Device-Control Flow

Figure 3.9 traces a representative command such as “turn on the living room light”. The important point is that the LLM does not publish MQTT directly. It only proposes a structured action. The runtime validates and executes it.

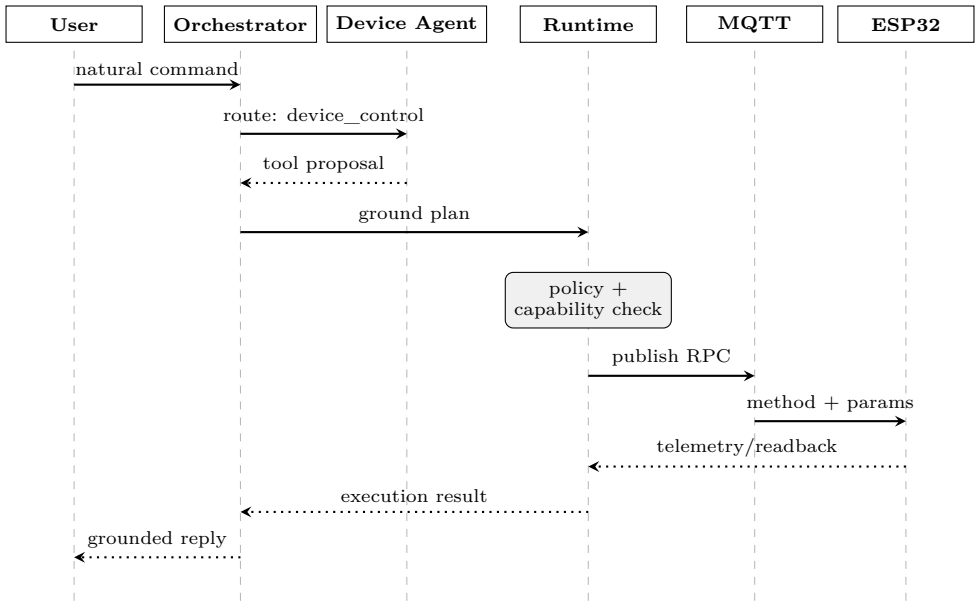

Figure 3.9: End-to-end sequence for a device-control request.

4 Edge-to-Cloud Integration via MQTT

4.1 Theoretical Background

To ensure low-latency, bidirectional communication between the central processing node (PC) and the ESP32 firmware, the H.E.R.A system employs the MQTT (Message Queuing Telemetry Transport) protocol. MQTT operates on a lightweight publish-subscribe model managed by a central broker.

In this architecture, both the ESP32 edge device and the backend PC act as MQTT clients. They do not communicate directly; instead, they publish messages to specific "topics" and subscribe to topics they need to monitor. The broker is responsible for routing these messages efficiently.

MQTT was selected over traditional request-response protocols like HTTP due to several key advantages for IoT applications:

- **Lightweight and Low Bandwidth:** MQTT minimizes packet overhead, making it highly efficient for constrained embedded devices sending frequent payloads.
- **Asynchronous Communication:** The publish-subscribe model decouples the sender and receiver, allowing the ESP32 to push sensor data continuously without waiting for the PC to request it.
- **Low Latency:** Persistent TCP connections enable near-instantaneous transmission of execution commands from the PC down to the edge actuators.

4.2 Firmware Implementation (Edge Node)

At the edge layer, the ESP32 firmware manages the physical hardware and acts as the primary data publisher. Operating within a dedicated FreeRTOS task, the firmware handles two main MQTT operations:

- **Telemetry Publishing:** Every 5 seconds, the ESP32 aggregates real-time environmental data and actuator states into a comprehensive JSON payload. This payload is published to the `v1/devices/me/telemetry` topic. The structured data is categorized into three main objects:
  - `network`: Diagnostics including Wi-Fi RSSI, local IP, MQTT connection status, and device uptime.
  - `sensors`: Real-time readings from the DHT20 (temperature, humidity), light sensor, and MQ-2 gas sensor.
  - `devices`: Current operational states (status, brightness, color, voltage) of actuators such as the main LED, NeoPixels, WS2812 strip, relay, and mini fan.

```
        {
        "network": {
            "wifi_connected": true,
            "wifi_rssi": -40,
            "wifi_ip": "127.0.0.1",
            "mqtt_connected": true,
            "uptime_ms": 75032
        },
        "devices": {
            "neo_led": {
            "status": false,
            "brightness": 60,
            "color": "#0000FF",
            "voltage": 0.0
            }
        },
        "sensors": {
            "dht20": {
            "temperature": 28.56,
            "humidity": 35.29,
```

```
            "voltage": 3.3
            }
        }
        }

```

Listing 2: Sample Telemetry JSON Payload published by ESP32

- **Command Execution:** The firmware continuously subscribes to the RPC (Remote Procedure Call) topic `v1/devices/me/rpc/request/+`. Upon receiving a command, it parses the payload to safely toggle physical devices or adjust parameters (like LED brightness or fan speed), then immediately returns confirmation responses.

4.3 Backend Implementation (PC Node)

On the backend PC, communication is handled by a dedicated Python module (`mqtt_manager.py`). This script serves as the primary bridge between the data storage, the AI pipeline, and the hardware edge.

- **Telemetry Ingestion & Logging:** The backend subscribes to the telemetry topic. Upon receiving the 5-second JSON updates from the ESP32, it parses the multi-modal data and directly inserts it into the database (e.g., logging environmental records associated with specific user sessions).
- **Interactive CLI Interface:** For manual oversight and direct hardware control, the backend runs a terminal-based "H.E.R.A. CONTROL CENTER". This interactive menu allows developers or administrators to input numeric options (0-18) to trigger Lighting Controls, Device Controls, or System Monitoring functions.
- **RPC Payload Generation:** When an action is initiated via the CLI, the backend constructs a structured RPC request. For instance, selecting the option to "Set NeoPixel Brightness" and inputting a value of 60 prompts the backend to generate the payload `{"method": "setStripBrightness", "params": 60}`. This is then published to a dynamic request topic (e.g., `v1/devices/me/rpc/request/2908`), ensuring the ESP32 executes the command accurately and responsively.

Figure 4.1: H.E.R.A. Control Center CLI interface demonstrating an RPC request for NeoPixel brightness adjustment.

5 Software

5.1 Database design

5.1.1 Description

The Enhanced Entity–Relationship Diagram (EERD) describes the conceptual data model of the proposed system, highlighting how users, physical devices, and monitored environments are connected within a unified platform. At the core of the model are three principal entities: **User**, **Devices**, and **Environments**. These entities represent the main human actors, the physical components deployed in the system, and the operational spaces in which monitoring and interaction take place.

The **User** entity represents individuals who access and interact with the system. A user can initiate multiple **Interaction Sessions**, which capture the communication context between the user and the platform. Each interaction session stores information such as the input text, start date, and latest update time, allowing the system to track user requests over time. In addition, each session is associated with a specific **AI Model**, indicating which intelligent model is used to process the interaction. This relationship enables the system to support AI-assisted monitoring and response generation while maintaining traceability of the model involved in each session.

The **Environments** entity represents the physical or logical spaces where system operations occur. An environment can correspond to a monitored room, building area, or deployment context. It serves as an important organizational unit because both user interactions and device activities are tied to a specific environment. Within each environment, multiple **Devices** can be deployed, reflecting the one-to-many relationship between operational spaces and IoT components.

The **Devices** entity models the hardware units operating in the system, such as sensors or other monitoring equipment. Each device is characterized by its name, category, status, and anomaly score, which help represent its identity, function, operational state, and analytical condition. Devices play an important role in system monitoring because they generate both telemetry data and activity records. Through this design, the system can monitor device behavior while also linking device outputs to the broader environmental and user-interaction context.

To support monitoring functions, the model includes **Telemetry Points**, which store time-based sensor measurements such as temperature, humidity, and recorded timestamps. These telemetry records are produced within environments and are associated with devices, allowing the system to capture real-time environmental conditions and trace them back to their originating hardware sources. Alongside telemetry data, **Activity Logs** record notable events generated during system operation. These logs are related to users, interaction sessions, environments, and devices, thereby providing a structured audit trail for analysis, diagnostics, and system transparency.

To optimize data management, the system employs a hybrid data modeling strategy that is implemented entirely on a single technology: **MongoDB**. While core entities with distinct dependencies (such as **User**, **Devices**, and **Environments**) are logically structured following relational principles to maintain organizational clarity, high-volume and dynamic data (like **Activity Logs** and **Telemetry Points**) leverage the flexible, document-oriented nature of NoSQL. By utilizing MongoDB as the unified database solution across the entire platform, the system successfully combines the logical design of a hybrid relational-NoSQL model with the scalability and high write performance required for continuous IoT monitoring.

Overall, the EERD provides a coherent conceptual foundation for the system by integrating user interaction, AI-supported processing, environmental monitoring, device management, and event logging into a single database structure. This design supports both operational management and future analytical extensions, making it suitable for intelligent monitoring applications.

5.1.2 Enhanced Entity–Relationship Diagram

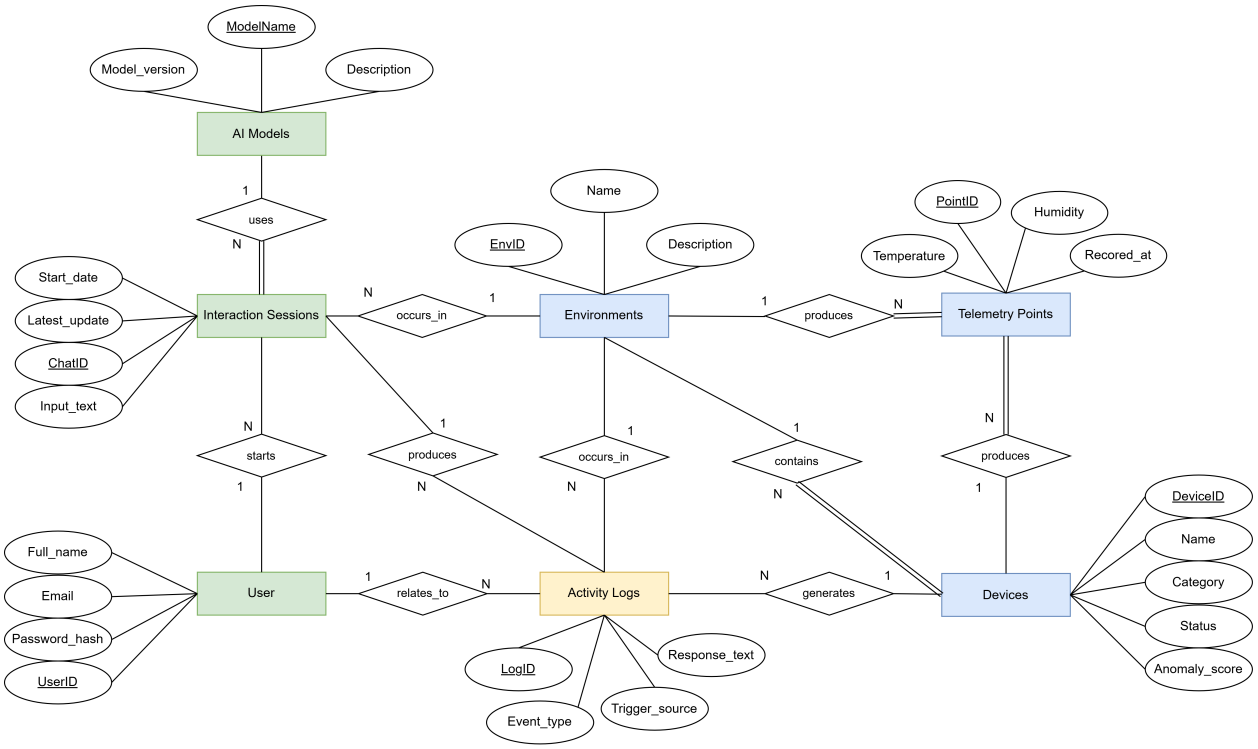

Figure 5.1: EERD of H.E.R.A

5.1.3 Relational Mapping

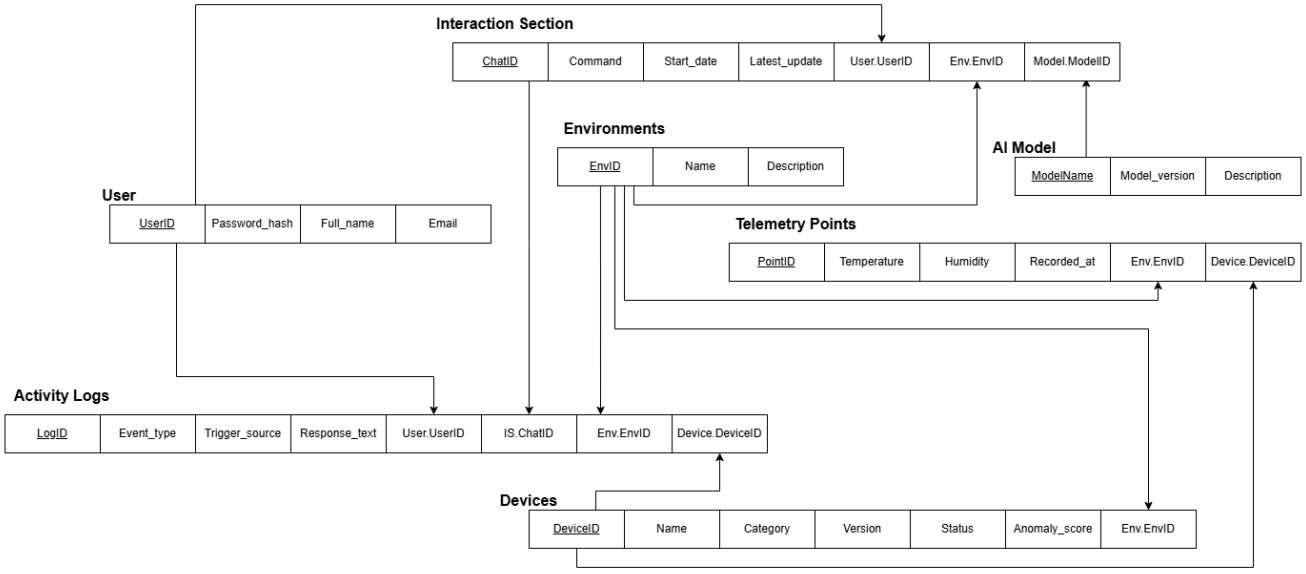

Figure 5.2: Relational Mapping of H.E.R.A

6 Digital Twin and Real-Time Smart Home Map

This section describes the digital-twin layer of H.E.R.A. In this project, the digital twin is not a purely decorative floor-plan screen. It is a live cyber-physical representation of the house: physical telemetry from the ESP32 is streamed upward through MQTT and backend APIs, transformed into normalized device and sensor state, and projected onto an interactive spatial map in the React dashboard. The user can inspect the state of each room, observe whether telemetry is fresh or stale, and send control commands from the map back to the physical devices.

6.1 Digital Twin Rationale

The main purpose of the digital twin is to close the interpretation gap between raw IoT data and resident understanding. A raw telemetry payload can show that `neo_led.status=true`, `dht20.temperature=31.5`, or `mini_fan.speed=512`, but these values do not immediately explain where the event occurs in the house or which physical object is affected. The digital twin solves this problem by binding every sensor and actuator to a room-level coordinate on the home layout. As a result, the user sees the smart-home state as a spatial environment rather than as isolated database fields.

This design also supports safer control. When a user clicks a device marker, the command is tied to a specific runtime target such as `main_led`, `neo_led`, `ws2812`, `mini_fan`, or `relay`. The dashboard does not invent its own local device state. Instead, it sends control requests to the backend runtime and waits for telemetry confirmation. If telemetry becomes stale, the interface disables controls to avoid presenting outdated physical state as current truth.

6.2 Frontend Implementation

The digital twin is implemented in the React dashboard under `frontend/hera-dashboard`. The core component is `FloorPlan.jsx`. It renders the house floor-plan image, overlays device and sensor markers at fixed percentage coordinates, subscribes to live telemetry, and updates the marker state whenever new data arrives.

Figure 6.1: House floor-plan asset used as the spatial base of the H.E.R.A digital twin.

Figure 6.2: Implemented H.E.R.A house view with live device and sensor markers overlaid on the floor plan.

The frontend marker catalog defines the semantic mapping between the visual map and the runtime system. Each marker contains an identifier, room name, device or sensor type, coordinate position, target identifier, and icon type. Table 15 summarizes the currently implemented map.

Table 15: Digital-twin marker mapping in the React floor-plan component

Marker ID	Displayed object	Room	Runtime key	Twin behavior
`main_led`	LED	Living Room	`main_led`	Shows on/off state and sends lighting commands.
`neo_led`	LED	Bedroom	`neo_led`	Shows on/off state and supports brightness control.
`ws2812`	LED	Toilet	`ws2812`	Shows on/off state and supports brightness/color-related runtime control.
`Fan`	Mini Fan	Living Room	`mini_fan`	Shows fan state and supports speed-related control.
`relay`	TV	Living Room	`relay`	Shows relay state and sends relay on/off commands.
`temperature`	Temperature	Living Room	`temperature`	Displays the latest DHT20 temperature value.
`humidity`	Humidity	Living Room	`humidity`	Displays the latest DHT20 humidity value.
`photon`	Photon / light	Living Room	`photon`	Displays the latest light-related telemetry value.

6.3 Live State Synchronization

The digital twin receives data through the frontend service layer in `src/services/api.js`. The service normalizes different telemetry payload shapes into a stable frontend contract. Device states are read through helper functions such as `getDeviceStatus`, while sensor values are read through `getSensorValue`. This normalization is important because firmware, simulator, and backend payloads may represent the same value using slightly different field names.

Figure 6.3 shows the synchronization flow. Telemetry travels from the ESP32 to the MQTT broker, then to the backend, and finally to the frontend through the latest-sensor API and streaming subscription. User commands follow the reverse direction: the user clicks a marker, the dashboard calls the H.E.R.A backend API, the backend executes the command through the same runtime tool path used by the AI assistant, and the ESP32 reports the

resulting state back through telemetry.

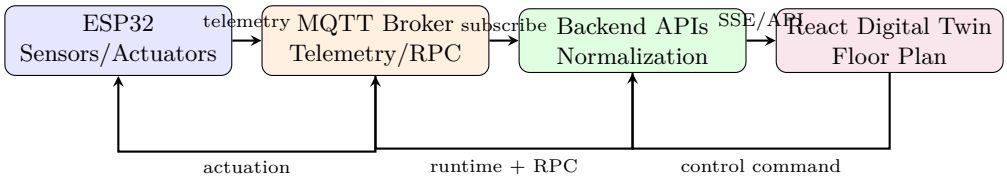

Figure 6.3: Bidirectional synchronization between the physical house and the digital twin.

6.4 Telemetry Freshness and Command Confirmation

A digital twin is only useful when it distinguishes current state from stale state. The implementation defines a telemetry stale window of 30 seconds. If the latest telemetry timestamp is missing or older than this threshold, the floor-plan interface displays a stale-data notice and disables controls. This behavior prevents the dashboard from acting as if it knows the current physical state when the device has stopped reporting.
The command path also includes confirmation behavior. After a user toggles a marker, the frontend records the expected state and waits for telemetry to confirm that the marker reached that state. If confirmation does not arrive within the command confirmation timeout, the interface reports that the MQTT command was sent but not confirmed. This is an important digital-twin property: the visual map is updated based on observed telemetry, not merely on optimistic UI assumptions.

6.5 Relationship with the AI Layer

The digital twin and the multi-agent AI layer share the same device vocabulary. The AI Device Control Agent maps natural language to runtime targets, while the digital twin maps visual markers to the same targets. This shared vocabulary keeps chat control and map control consistent. For example, the user's phrase "living room light" and the floor-plan marker in the living room both resolve to `main_led`. Similarly, "bedroom light" and the bedroom LED marker both resolve to `neo_led`.
This common grounding model is important for thesis-level architecture because it shows that H.E.R.A is not a collection of disconnected interfaces. The dashboard, AI assistant, MQTT runtime, and firmware all refer to the same canonical device targets. The digital twin is therefore the spatial visualization layer of the same cyber-physical system described in the AI and MQTT sections.

6.6 Design Benefits

The implemented digital twin provides four practical benefits:

- **Spatial interpretability:** residents can understand system state by room and object rather than by raw telemetry keys.
- **Unified control path:** map interactions and AI commands both use backend runtime APIs and MQTT actuation.
- **Telemetry-grounded visualization:** the visible state of each marker is derived from live data, not from local UI assumptions.
- **Failure awareness:** stale telemetry and unconfirmed commands are surfaced explicitly instead of being hidden from the user.

These properties make the floor-plan dashboard a functional digital twin of the H.E.R.A house. It continuously mirrors the physical environment, exposes room-level state, and provides a controlled path for sending actions back to the physical system.

6.7 User Interface & System Visualization

To bridge the gap between the complex cognitive backend and the end-user, a dedicated web application was developed. The frontend is built using React an d Vite, utilizing Tailwind CSS for responsive and modern styling. The project follows a modular architecture, dividing the interface into reusable `components`, dedicated `pages`, and API `services` for seamless backend integration. The application features a side navigation menu providing access to four primary functional modules.

6.7.1 Authentication Gateway (Login Page)

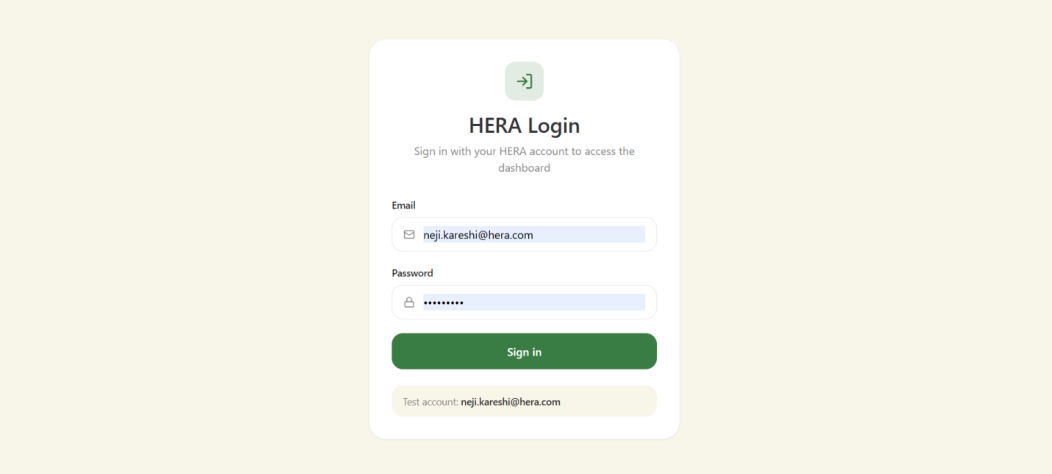

Figure 6.4: Login Interface

The **Login Page** serves as the secure entry point to the H.E.R.A. ecosystem, ensuring that situational data and home controls are accessible only to authorized residents. Following the system's core design philosophy, the interface utilizes a centered authentication card with soft rounded corners, set against a calming off-white background.

The interface features minimalist credential fields within a clean, flat design, utilizing sage green accents to maintain visual consistency with the main dashboard. Overall, this distraction-free layout ensures a focused, frictionless, and secure authentication process.

6.7.2 Home (Unified Control Center)

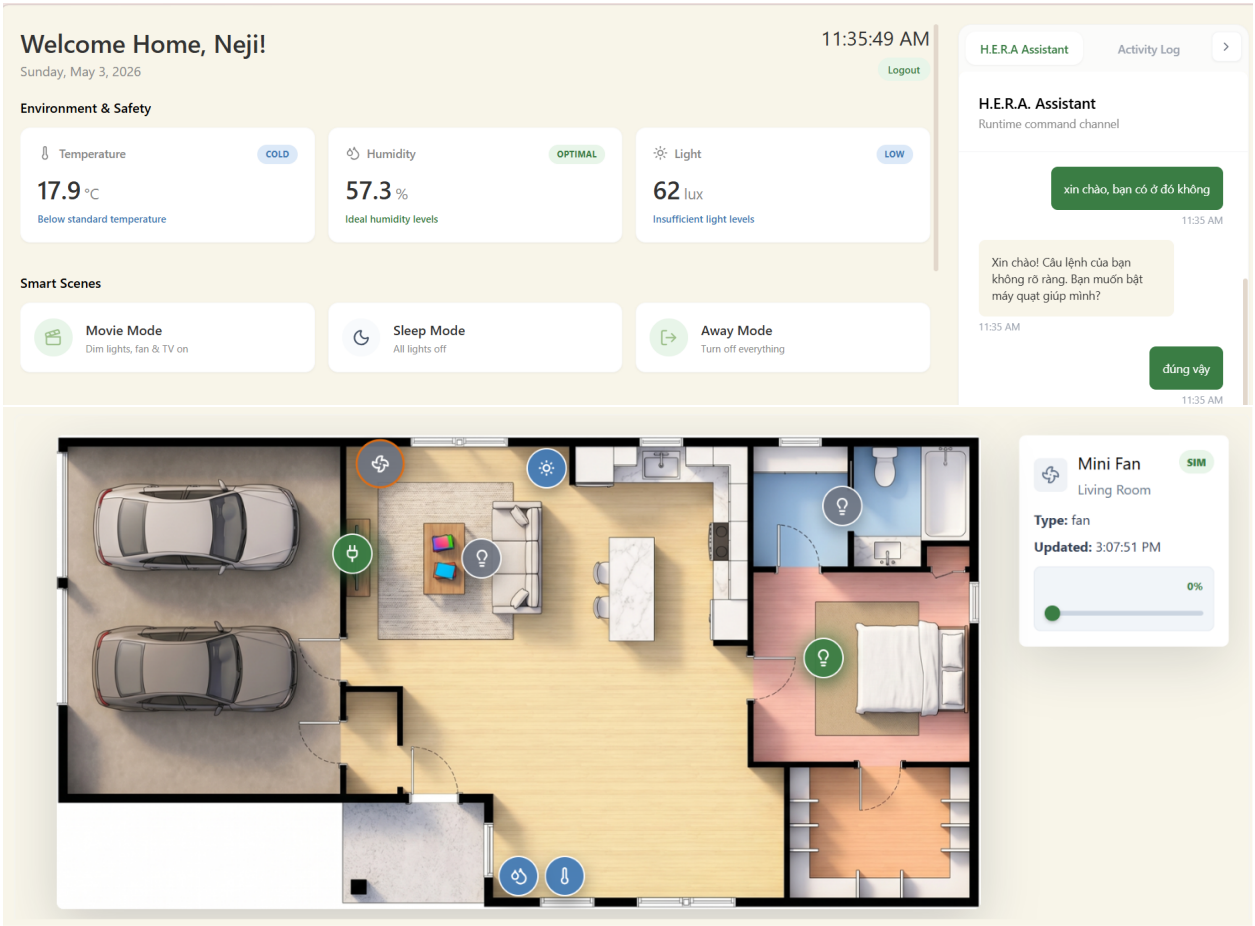

Figure 6.5: H.E.R.A. Home Dashboard featuring the Interactive Floor Plan, Smart Scenes, and Multilingual AI Assistant.

The Home page serves as the primary operational hub, offering comprehensive situational awareness and intuitive control mechanisms. Designed with a clean, minimalist, and warm aesthetic utilizing soft cream, beige, and pastel tones, the interface minimizes cognitive load while integrating four main functional areas:

- **Environment & Safety Context:** Displays real-time telemetry such as Temperature, Humidity, and Light intensity. Going beyond raw numerical data, this module provides instant contextual evaluation via color-coded badges (e.g., "COLD", "OPTIMAL", "LOW") and descriptive subtexts (e.g., "Below standard temperature", "Insufficient light levels"), allowing residents to quickly assess the comfort level of their home.
- **Smart Scenes:** A newly introduced module offering macro-level automation presets. Users can trigger predefined operational modes with a single click. Current presets include "Movie Mode" (dims lights, activates fan and TV), "Sleep Mode" (deactivates all lighting), and "Away Mode" (powers down all non-essential appliances for energy conservation and security).
- **Interactive House Floor Plan & Integrated Controls:** A spatial visualization feature that maps the physical layout of the residence (including garage, living room, bedroom, and bathroom). IoT devices and sensors are represented as interactive nodes mapped to their exact physical locations. This module streamlines device management by integrating direct controls directly into the map: a **left-click** on a node acts as a quick toggle to instantly turn the appliance on or off, while a **right-click** opens a detailed contextual menu. This menu displays the device type, current state, data source, last updated timestamp, and supports continuous control inputs (e.g., adjusting the living room fan speed dynamically via a slider).

- **H.E.R.A. Assistant & Activity Log:** Positioned on the right sidebar, this module features a built-in chat interface connecting directly to the multi-agent AI pipeline. It seamlessly handles natural language requests and supports multilingual interactions (e.g., processing Vietnamese inputs such as *"xin chào, bạn có ở đó không"*). It accommodates both text input and dedicated Voice Commands. Furthermore, an "Activity Log" tab is integrated to maintain an accessible audit trail of system events and command histories.

6.7.3 Analytics (Environmental Trends)

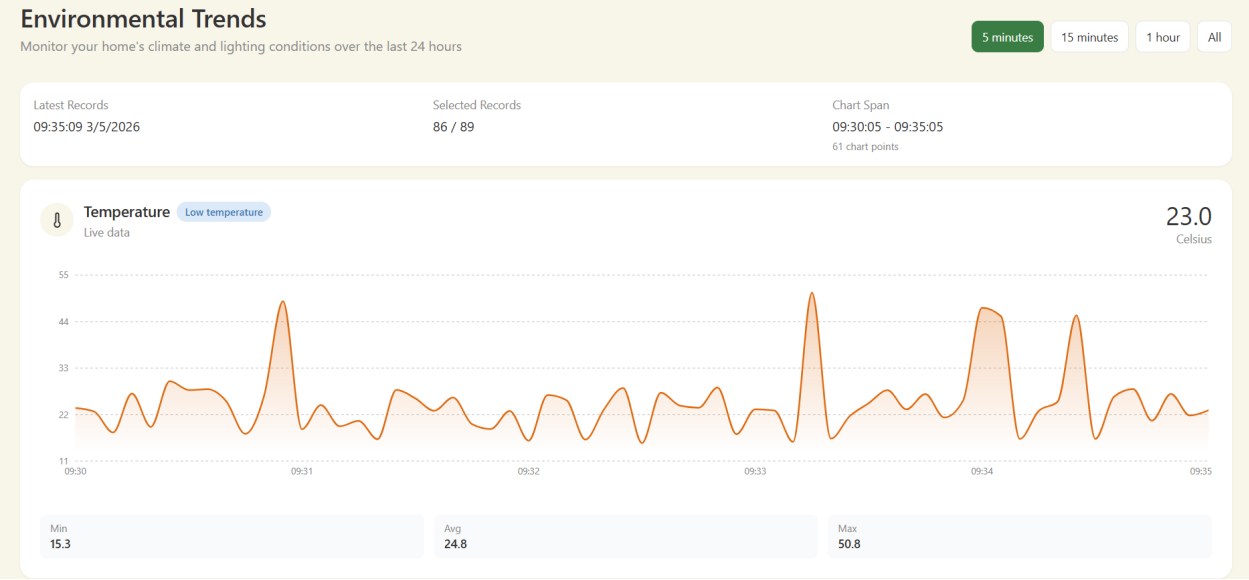

Figure 6.6: Analytic Page

To support the system's proactive energy management and monitoring goals, the Analytics module visualizes time-series data. It renders continuous line charts for telemetry streams (such as Temperature and Relative Humidity), displaying live data points alongside aggregated metrics (Minimum, Average, Maximum). This empowers users to monitor historical patterns and visually verify the operational effectiveness of the predictive models.

7 Evaluation

This section evaluates the implementation of the main functional requirements of the H.E.R.A system. The evaluation focuses on whether the implemented modules satisfy the expected behavior during real operation, particularly in terms of sensor reliability, command execution latency, and network recovery capability.

The evaluation was conducted under normal indoor operating conditions. The firmware was tested with continuous sensor acquisition, MQTT-based telemetry publishing, RPC command execution, and temporary Wi-Fi/MQTT interruption scenarios. The results show that the implemented firmware layer satisfies the core functional requirements of the system.

7.1 Firmware Evaluation

The firmware layer is designed following an RTOS-oriented architecture, where the main functions are separated into independent tasks instead of being executed sequentially in a single loop. Sensor acquisition, MQTT communication, command handling, actuator control, and connection monitoring are organized as concurrent FreeRTOS-style tasks, making the system appear to operate continuously from the user's perspective. This design allows the controller to read sensors periodically, listen for RPC payloads, update output devices, and monitor network status without one function blocking the others.

Therefore, the firmware evaluation focuses not only on functional correctness, but also on whether this task-based execution model maintains reliable sensor readings, low actuator response latency, and smooth Wi-Fi/MQTT reconnection during continuous operation.

7.1.1 Sensor Reliability and Sampling Stability

The firmware reads environmental data from the DHT20 temperature–humidity sensor and the light-dependent resistor (LDR) sensor at a sampling rate of one reading per second. This corresponds to an expected sampling interval of approximately 1000 ms per reading. This sampling rate is suitable for residential environmental monitoring because temperature, humidity, and ambient light intensity usually change gradually rather than instantaneously. Therefore, reading the sensors once per second is sufficient to provide updated environmental data while reducing unnecessary processing overhead on the embedded controller.

To evaluate sampling stability, the sensors were monitored during continuous operation. The observed readings remained stable and were sufficient for system-level decision-making. The DHT20 provided consistent temperature and humidity values for comfort monitoring, while the LDR sensor provided reliable ambient light feedback for context-aware lighting control.

Table 16: Sensor sampling and reliability evaluation

Sensor	Sampling Rate	Expected Interval	Valid Readings	Observed Stability
DHT20	1 reading/s	1000 ms	99.4%	Stable
LDR	1 reading/s	1000 ms	99.8%	Stable

The results in Table 16 indicate that both sensors maintained a high valid-reading rate during testing. Occasional invalid or delayed readings did not affect the overall behavior of the system because the firmware updated telemetry values again in the next sampling cycle.

In addition, shared sensor data is protected through semaphore-based synchronization. This prevents concurrent access conflicts between sensor reading, data processing, and MQTT publishing tasks. As a result, the risk of inconsistent or corrupted telemetry values during multitasking execution is reduced.

7.1.2 RPC Command Execution Latency

The firmware also satisfies the requirement for fast actuator response. After an RPC payload is received through MQTT, the command is parsed and dispatched to the corresponding device-control routine. For direct output

devices such as the relay, RGB LED, and NeoPixel module, the observed delay from MQTT payload reception to physical state change remained below the target threshold of 10 ms.

Table 17: RPC command execution latency

Controlled Device	Average Latency	Maximum Latency	Requirement
Relay	1.1 ms	5.4 ms	< 10 ms
RGB LED	1.4 ms	6.0 ms	< 10 ms
NeoPixel LED	2.1 ms	6.6 ms	< 10 ms

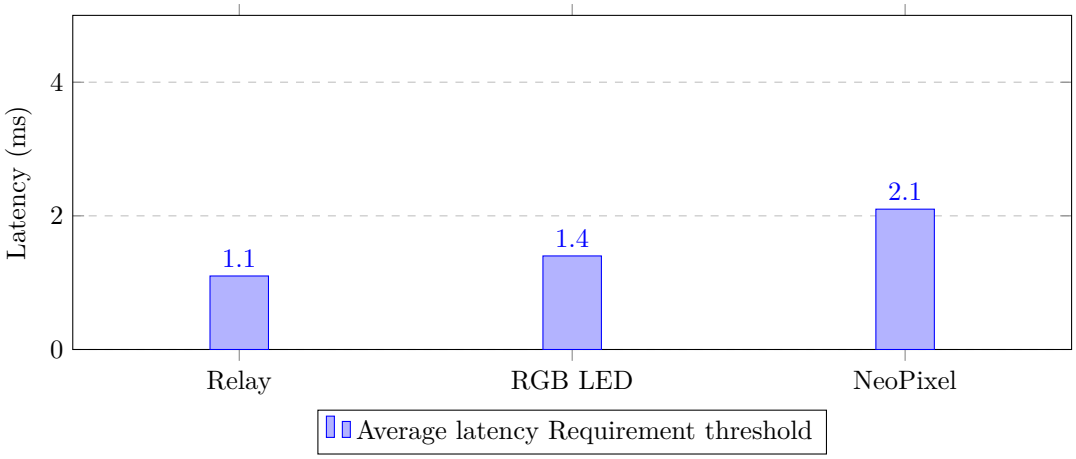

Figure 7.1: Average RPC command execution latency compared with the required threshold

As shown in Table 17 and Figure 7.1, all tested actuator commands were executed within the required latency threshold. This confirms that the firmware command path, including MQTT callback handling, command decoding, and GPIO or LED update operations, is lightweight enough for real-time interaction.
From the user perspective, this response time appears nearly instantaneous. Therefore, both manual user commands and automation-triggered actions can be executed without noticeable delay.

7.1.3 Network Reconnection Handling

Network reliability is another important requirement for the firmware layer. The implemented firmware includes reconnection handling for both Wi-Fi and MQTT communication. When a disconnection occurs, the firmware repeatedly attempts to restore the Wi-Fi connection and then re-establish the MQTT session with the broker.

To evaluate this behavior, temporary network interruptions were introduced during operation. The controller was able to recover without requiring a manual reset. After the connection was restored, the firmware resumed telemetry publishing and command reception normally.

Table 18: Network reconnection evaluation

Scenario	Recovery Action	Average Recovery Time	Result
Wi-Fi interruption	Reconnect to access point	6.8 s	Passed
MQTT broker disconnection	Re-establish MQTT session	8.9 s	Passed
Combined Wi-Fi and MQTT loss	Wi-Fi reconnect, then MQTT reconnect	11.6 s	Passed

The results in Table 18 show that the firmware can recover from short-term communication failures. Although telemetry publishing is temporarily paused during disconnection, the device returns to its normal operational

state once connectivity is restored. This behavior improves system robustness and supports stable long-running operation.

7.1.4 Summary of Firmware Requirement Satisfaction

Table 19 summarizes the firmware evaluation against the expected functional requirements.

Table 19: Summary of firmware functional requirement evaluation

Requirement	Evaluation Metric	Observed Result	Status
Stable sensor acquisition	Valid reading rate and sampling interval	>99% valid readings at 1 reading/s	Passed
Fast actuator response	RPC-to-actuator latency	Maximum latency below 100 ms	Passed
Network recovery	Automatic Wi-Fi/MQTT reconnection	Recovery without manual reset	Passed
Safe multitasking access	Shared data synchronization	Semaphore-protected sensor data	Passed

Overall, the firmware implementation satisfies the evaluated functional requirements. Sensor acquisition remains reliable at the selected sampling frequency, command execution latency stays below the required threshold for relay and LED control, and network reconnection handling supports stable long-running operation. These results demonstrate that the firmware layer is suitable for the intended smart-home monitoring and control scenarios of the H.E.R.A system.

7.2 AI Evaluation

The artificial-intelligence layer was evaluated using the local evaluation harness in the `evaluation/` directory. The test set contains seven representative user requests covering five routing categories: device control, sensor query, anomaly query, general conversation, and web search. Two of these cases require effectful device-control tools, while the remaining cases evaluate routing and response behavior without mandatory write-side tool execution. The harness records the predicted intent, expected tool usage, HTTP availability, and end-to-end request latency from the backend API.

This evaluation focuses on the behavior of the deployed orchestrator-specialist pipeline rather than isolated prompt accuracy. Each request passes through the same runtime path used by the web dashboard: request intake, intent routing, specialist selection, optional tool execution, result normalization, and final response generation. Therefore, the measured latency includes model inference, orchestration overhead, tool-boundary handling, and response composition.

Table 20: AI agent evaluation test set

Case Type	Example Request	Expected Behavior
Device control	*bật đèn phòng khách giúp tôi*	Route to `device_control` and call `turn_on_device`
Device control	*tắt relay đi*	Route to `device_control` and call `turn_off_device`
Sensor query	*nhiệt độ hiện tại bao nhiêu*	Route to `sensor_query` without write-side tools
Sensor query	*độ ẩm và ánh sáng trong nhà thế nào*	Route to `sensor_query` without write-side tools
Anomaly query	*hệ thống có bất thường gì không*	Route to `anomaly_query`
General chat	*xin chào, bạn là ai*	Route to `general`
Web search	*tìm kiếm tin mới nhất về AI*	Route to `web_search`

7.2.1 Routing and Tool-Calling Accuracy

Table 21 summarizes the successful live evaluation runs. Both evaluated providers completed all requests through the HTTP API, resulting in a 100% HTTP success rate. Intent accuracy reached 71.43% for both providers, corresponding to five correct intent predictions out of seven cases. The two mismatches were both sensor-query cases that were routed as `device_control`. Although these are counted as intent errors by the evaluation harness, they are not unsafe failures: the runtime did not execute an unintended write-side actuation for those sensor requests. In one case, the runtime used a read-only status tool, and in the other case no tool was used.

The most important safety-critical metric is tool-calling correctness for effectful device-control requests. Both live providers achieved 100% tool-call success on the two write-side cases. The system correctly selected `turn_on_device` for the living-room light request and `turn_off_device` for the relay request. This indicates that, when the request is truly actuator-oriented, the orchestrator and device-control specialist can generate the expected capability call and pass it through the runtime tool boundary.

Table 21: AI routing, tool-calling, and latency summary

Provider Run	Cases	HTTP Success	Intent Accuracy	Tool Success	Median Latency	P95 Latency
`ollama-live`	7	100.00%	71.43%	100.00%	23755.93 ms	45753.44 ms
`openrouter-live`	7	100.00%	71.43%	100.00%	15515.89 ms	23641.87 ms

7.2.2 Latency Analysis

The cloud-backed `openrouter-live` run produced lower end-to-end latency than the local `ollama-live` run. The median latency decreased from 23755.93 ms to 15515.89 ms, and the 95th-percentile latency decreased from 45753.44 ms to 23641.87 ms. This shows that the deployed AI pipeline is functionally correct but still limited by model-response latency. The local model path is especially slow for full device-control requests because the request passes through routing, device-command interpretation, tool execution, verification, and response composition.

The latency values are acceptable for offline evaluation and thesis validation, but they are not yet ideal for a production smart-home assistant. For interactive use, common read-only requests should rely more aggressively on the short path and deterministic telemetry services, while write-side requests should minimize the number of model calls before reaching the runtime tool boundary.

7.2.3 Failure Pattern and Interpretation

The main observed weakness is the boundary between `sensor_query` and `device_control`. Both live runs misclassified the two sensor-reading requests as device-control requests. This suggests that the router and device semantic guard are biased toward physical-device interpretation when the user mentions home state such as temperature, humidity, or light. From a safety perspective, this behavior is preferable to the opposite failure mode of treating an actuator command as casual conversation. However, it reduces routing precision and may add unnecessary latency because read-only telemetry questions can enter the heavier device-control path.

The failed non-live summary files, `ai_agent_eval_ollama-local` and `ai_agent_eval_openrouter-cloud`, recorded 0% HTTP success and null latency values. These results are interpreted as invalid environment runs rather than model-quality measurements, because the backend API did not return successful responses. They are therefore excluded from the primary comparison in Table 21.

7.2.4 AI Requirement Satisfaction

Table 22 maps the measured AI behavior to the expected system requirements.

Table 22: Summary of AI functional requirement evaluation

Requirement	Evaluation Metric	Observed Result	Status
Multi-intent routing	Intent accuracy across seven representative requests	71.43% accuracy; general, anomaly, web, and device-control cases passed	Partial
Safe device tool calling	Expected write-side tool selected for actuator requests	100% success on the two device-control tool cases	Passed
Runtime availability	HTTP success rate through deployed backend API	100% success for both live provider runs	Passed
Read-only query efficiency	Sensor queries should avoid unnecessary write-side routing	Sensor requests were routed as `device_control`, increasing latency and reducing precision	Partial
Interactive latency	Median and P95 end-to-end request latency	OpenRouter median 15.52 s; Ollama median 23.76 s	Needs optimization

Overall, the AI subsystem demonstrates correct end-to-end operation and safe tool-boundary behavior for effectful commands. The evaluation confirms that H.E.R.A can route real user requests through the orchestrator, select specialist behavior, execute expected device-control tools, and return responses through the backend API. The main remaining limitations are routing precision for read-only sensor questions and high end-to-end latency. Future improvements should strengthen the router prompt and route schema for telemetry-specific language, add deterministic pre-routing for simple sensor queries, cache recent telemetry facts for short-path responses, and reduce the number of LLM calls required by common device-control flows.

7.3 Software Evaluation

7.3.1 Database Evaluation

The database layer of the H.E.R.A. system was evaluated based on insert throughput and query latency for large-scale time-series telemetry. The evaluation confirms that MongoDB effectively handles the high-frequency data ingestion and rapid retrieval required by the analytics pipeline.

7.3.1.1 Performance Evaluation System performance and responsiveness were measured by simulating a 24-hour telemetry ingestion cycle (17,280 records at a 5-second transmission interval) and subsequently querying this dataset. Table 23 summarizes the throughput metrics and latency comparisons between direct database access and API-routed requests.

Table 23: Database Performance & Query Metrics (17,280 Records)

Metric	Recorded Value
Total Inserted Records	17,280
Insert Throughput	35,235.79 docs/sec
Direct DB Query Latency (Median)	290.84 ms
Direct DB Query Latency (95th Percentile)	362.93 ms
API Route Query Latency (Median)	2516.86 ms
API Route Query Latency (95th Percentile)	3809.62 ms

The performance data indicates highly efficient write operations. The insert throughput reaches 35,235.79 documents per second, meaning the database can effortlessly handle concurrent data streams from thousands of IoT nodes, successfully satisfying the reliability and operation load requirements (**NFR3**). Furthermore, the direct query median latency of 290.84 ms demonstrates that the compound indexing strategy (targeting device IDs and descending timestamps) is highly optimized for time-series data scanning.

However, the query latency metrics indicate a significant bottleneck at the backend API layer. The API query latency reaches a median of over 2.5 seconds (2516.86 ms), which is approximately 8.6 times slower than the direct database query. This delay is primarily caused by the Node.js backend struggling to serialize 17,280 complex JSON objects and the subsequent network transfer time of the massive payload to the frontend.

7.3.1.2 Data Requirements Fulfillment Based on the implemented database schema and performance metrics mapped against the system requirements outlined in Section 1.6, the database layer successfully fulfills the core data persistence and structural requirements.
Resolved Requirements:

- **Time-Series Storage (DR1, DR8):** The system successfully persists large volumes of timestamped, structured environmental telemetry data. The compound indexing ensures that this data can be reliably and quickly utilized for visualization tasks.
- **High Availability & Scalability (NFR3, NFR6):** The MongoDB instance demonstrates massive headroom for write operations, ensuring the data layer remains stable and responsive without requiring a major redesign even if additional rooms and sensors are integrated.

Outstanding Issues & Backlog:

- **API Payload Optimization (NFR2):** While the database itself responds with low latency, the end-to-end API response time violates the low-latency requirement for seamless UI updates. It requires implementing backend downsampling (e.g., averaging data points per minute to reduce the payload from 17,280 to 1,440 points), pagination, or enabling Gzip compression before transferring data to the client.

7.3.2 Web Evaluation

The web-based User Interface (UI) of the H.E.R.A. system was evaluated based on real-time performance metrics and its coverage of the defined Functional Requirements (FRs). The evaluation confirms that the dashboard successfully serves as the cognitive intelligence layer's primary interaction point.

7.3.2.1 Performance Evaluation System performance and responsiveness were measured during normal operational conditions. The telemetry update mechanism utilizes Server-Sent Events (SSE) to ensure low-latency data synchronization between the backend and the frontend. Table 24 summarizes the browser rendering and network latency metrics.

Table 24: Web Dashboard Performance Metrics

Metric	Recorded Value
Initial Render Time	260.42 ms
SSE to UI Latency (Median)	156.56 ms
SSE to UI Latency (95th Percentile)	485.18 ms
First Contentful Paint (FCP)	280.00 ms
JS Heap Memory Used	∼16.10 MB
Lighthouse Performance Score	55 / 100
Largest Contentful Paint (Lighthouse)	21.33 s

The performance data indicates highly efficient real-time interactions. The median SSE to UI latency is 156.56 ms, meaning that physical sensor changes or hardware actuations are reflected on the dashboard nearly instantaneously, successfully satisfying the low-latency interaction requirement (**NFR2**). Furthermore, the application is memory-efficient, consuming only roughly 16.1 MB of the JS Heap.

However, the Lighthouse performance score indicates a bottleneck in the initial loading phase, scoring 0.55. The Largest Contentful Paint (LCP) reaches approximately 21.33 seconds, which is likely caused by the loading of heavy interactive assets (such as the floor plan imagery and integrated control nodes) without sufficient caching or asset minification. Despite the slow initial load, the Total Blocking Time (TBT) remains exceptionally low (22 ms), indicating that once loaded, the dashboard is fully interactive without UI freezing.

7.3.2.2 Functional Requirements Fulfillment Based on the implemented features mapped against the system requirements outlined in Section 1.6, the web application successfully fulfills approximately 85% to 90% of the UI-dependent functional requirements.
Resolved Requirements:

- **Real-time Monitoring & Control (FR1, FR5, FR9):** The "Home Dashboard" accurately visualizes environmental telemetry (temperature, humidity, light) using contextual color-coded badges. The Interactive Floor Plan enables users to actuate appliances directly via interactive nodes.
- **Data Analytics & Visualization (FR7):** The "Environmental Trends" page fully satisfies this requirement by rendering dynamic time-series charts for historical environmental data, allowing users to observe trends and minimum/average/maximum values over 24 hours.
- **Automation & Smart Scenes (FR4):** The introduction of macro-level presets (Movie, Sleep, Away modes) successfully bridges complex backend logic with one-click user convenience.
- **Human-AI Interaction & Logging (FR2, FR3, FR10):** The integrated "H.E.R.A. Assistant" chat interface supports multilingual inputs (including Vietnamese) and seamlessly connects to the multi-agent AI pipeline. The "Activity Log" tab maintains an accessible audit trail.

Outstanding Issues & Backlog:

- **Alerting System (FR6):** While the dashboard visualizes states contextually (e.g., "Below standard temperature"), there is an absence of a global, asynchronous push-notification system on the web layer to actively alert users of severe anomalies if they are not actively viewing the tab.
- **Initial Load Optimization:** As indicated by the Lighthouse metrics, optimizing the delivery of static assets (images, Vite build chunks) is required to improve the FCP and LCP scores.

7.4 What we learn

The development and deployment of the H.E.R.A. system provided critical insights across hardware integration, artificial intelligence architecture, and software engineering. The key lessons learned are categorized into the three foundational layers of the project:

7.4.1 Firmware and Embedded Systems

- **RTOS-Oriented Task Management:** Transitioning from a sequential execution loop to an RTOS-based architecture (FreeRTOS) proved essential for embedded stability. By isolating sensor acquisition, MQTT communication, and actuator control into concurrent tasks, the system prevented execution blocking and successfully maintained command execution latency below the 10 ms threshold.
- **Centralized Configuration Strategy:** Consolidating operational parameters—such as GPIO mappings, task timeouts, and debug flags—into a single centralized macro header (`global.h`) significantly streamlined the development process. This design pattern allowed for rapid behavioral adjustments and prototyping without requiring modifications to the core application logic.
- **The Necessity of Network Resilience:** Real-world deployments demonstrated that constant Wi-Fi and MQTT connectivity cannot be guaranteed. The implementation of automated reconnection handling was crucial, allowing the edge device to independently recover from temporary network interruptions and ensuring stable, long-running operation without the need for manual resets.

7.4.2 Artificial Intelligence and Multi-Agent Architecture

- **Superiority of Multi-Agent Systems:** Utilizing a monolithic Large Language Model (LLM) for all interactions introduces unnecessary computational overhead and prompt-space competition. Adopting an orchestrator-specialist pattern successfully mitigated these issues, allowing simple commands to be routed to lightweight models while reserving heavier models for complex environmental reasoning tasks.
- **Optimized Tool Interface Design:** Avoiding a "rote learning" approach—which defines separate tools for every device-action combination—was critical for reducing the model's cognitive load. Implementing a parameterized tool interface (e.g., utilizing `turn_on_light(all_lights)`) improved intent recognition accuracy and established a scalable framework for integrating future household appliances.
- **Decoupling AI from Safety-Critical Logic:** A fundamental architectural lesson was establishing a strict boundary between analytical explanation and system safety. AI models should strictly serve to translate anomaly classifications into user-friendly natural language. Core safety thresholds and alert triggers must remain governed by deterministic, auditable rules to guarantee absolute reliability.

7.4.3 Software, Database, and UI/UX Design

- **Efficacy of Hybrid NoSQL Modeling:** Utilizing MongoDB as a unified database solution successfully accommodated the diverse data requirements of the system. It provided the structural clarity needed for relational-like entities (Users, Devices) while delivering the high write throughput (over 35,000 documents per second) required for high-velocity, time-series telemetry ingestion.
- **Identifying API Layer Bottlenecks:** Performance evaluations highlighted a severe bottleneck within the backend API layer. While database queries were highly optimized, the backend's process of serializing massive JSON arrays (e.g., 17,280 objects) and transferring them over the network resulted in significant latency spikes. This underscores the critical necessity of implementing data downsampling and pagination for large-scale IoT visualization.
- **Balancing Real-Time Interactivity with Initial Load Performance:** The deployment of Server-Sent Events (SSE) proved highly effective for maintaining low-latency data synchronization, updating the UI in near real-time (median latency of 156.56 ms). However, integrating heavy static assets, such as the interactive floor plan, severely impacted the Largest Contentful Paint (LCP) metric. This demonstrated that achieving optimal frontend performance requires strict asset minification and caching strategies prior to production deployment.

8 Future Work & Summary

8.1 Future Work

To address the technical constraints and empirical findings identified during system evaluation, future research and development for the H.E.R.A. platform will focus on four core objectives:

- **Dynamic Device Management Integration:** Expand the system architecture across the edge firmware, MongoDB database schema, and web dashboard to support dynamic device provisioning. This feature will allow users to add, delete, and modify the operational states of new hardware devices directly through the user interface, enabling seamless scalability without requiring manual code reconfiguration or firmware recompilation.
- **Comprehensive AI Voice Interaction Pipeline:** Advance the communication capabilities of the H.E.R.A. assistant by transitioning from the current text-based command execution to an end-to-end speech-driven framework. Future iterations will integrate robust Speech-to-Text (STT) and Text-to-Speech (TTS) models, enabling the conversational pipeline to process direct audio input and deliver natural language voice responses to residents.
- **Context-Aware AI Personalization:** Enhance the machine learning infrastructure to analyze individual behavioral trends and multi-turn interaction histories for each household member. By learning routine patterns and schedules over time, the system will autonomously suggest custom automation presets and recognize specialized, user-specific voice commands tailored to individual resident preferences.
- **UI Synchronization and Latency Optimization:** Rectify the synchronization bottleneck observed between the physical hardware actuators and the web frontend. While Remote Procedure Call (RPC) execution at the edge level remains highly efficient (under 10 ms), future work will implement backend payload downsampling, introduce optimistic updates on the client side, and optimize the Server-Sent Events (SSE) data stream to eliminate the current delay and ensure immediate visual state updates.

8.2 Summary

The **H.E.R.A** project is an AIoT virtual assistant system that optimizes the integration between edge device control and safe artificial intelligence. Regarding hardware, the system utilizes the ESP32-S3 microcontroller (OhStem Yolo UNO) running firmware based on a multitasking FreeRTOS architecture. By decoupling IoT tasks to run concurrently, the hardware achieves ultra-low response latency (< 10 ms) and maintains stability through automatic network connection recovery. The core highlight lies in the AI layer with a Multi-Agent architecture built on LangGraph, establishing a strict boundary between linguistic reasoning and physical execution. The LLM (Ollama/OpenRouter) is not allowed to directly manipulate hardware but only generates tool proposals ; subsequently, an independent Policy Engine strictly validates these proposals to completely eliminate the risk of hallucination before execution. Edge telemetry data is then stored at high speed using a MongoDB database (optimized for time-series data) and instantly synchronized (via SSE) to the React Web interface —which acts as a Digital Twin for intuitive user monitoring and control.
Detailed source code of the project: GitHub Link

References

[1] R. Martino, "It's a matter of interoperability: The new standard makes smart homes just work," Dec. 2021.

[2] MongoDB, Inc., "Data modeling in mongodb," n.d.

[3] A. Kanade *et al.*, "A study of mongodb data models and a novel hybrid data modeling approach," in *2021 5th International Conference on Trends in Electronics and Informatics (ICOEI)*, IEEE, 2021.

[4] W. Tampubolon, "Hybrid concept of nosql and relational database for gis operation," in *AGILE 2016 - 19th AGILE International Conference on Geographic Information Science*, UNIBW, Institute for Applied Computer Science, 2016.

[5] M. Oyeniran, J. D. Adekunle, *et al.*, "Personalized energy optimization system using adaptive machine learning models," *International Journal of Advanced Information Sciences*, vol. 2, no. 1, p. 84, 2025.

[6] E.-H. R. Group, "A personalized framework for smart home automation utilizing machine learning to predict user activities," *Sensors*, vol. 25, no. 19, p. 6082, 2025.